«Año de la Esperanza y el Fortalecimiento de la Democracia»

Universidad Tecnologica del Perú

Curso: Herramientas de Desarrollo

SISTEMA DE GESTION PARA UN RESTAURANTE

Profesor: Jose Luis Milla Flores

Alumnos:

-Parejas Marcelo, Roosevelt Joaquin

-Collana Perez, Maria Fernanda

-Bayona Cobeñas, Cristopher Franchesco

-Muñoz Zapata, Loana Aracelli

2026

Universidad Tecnológica del Perú

Tabla de contenido

1. Realidad Problematica 9

2. Justificación del Proyecto 10

3. Objetivos 10

3.1. Objetivo General: 10
3.2. Objetivos Específicos: 11
4. Marco teórico: 12

4.1. Sistemas de Gestión de Restaurantes 12
4.2. Gestión de inventarios 12
4.3. Automatización de procesos 12
4.4. Sistemas de información 13
5. Tecnologías utilizadas en el desarrollo 14

5.1. Spring boot 14
5.2. MySQL 14
5.3. GitHub 14
5.4. Ingeniería de software 14
6. Metodología 15

6.1. Tipo de investigación 15
6.2. Enfoque de investigación 15
6.3. Diseño de investigación 15
6.4. Metodología de desarrollo 15
6.5. Herramientas tecnológicas 16
7. Requisitos 17

7.1. Requisitos Funcionales 17
7.2. Requisitos No Funcionales 17
7.3. Restricciones Técnicas 17
7.4. Alcance del Sistema 17
8. Diagramas 19

8.1. Diagrama Entidad-Relación (ER) 19
8.2. Diagrama de Casos de Uso 21
8.3. Actor: Mesero 22
8.3.1. CUS01 - Registrar Pedido 22
8.3.2. CUS02 - Gestionar Mesa 22
8.3.3. CUS03 - Gestionar Pedido 23
8.3.4. CUS04 - Consultar Pedido 23
8.4. Actor: Administrador 23
8.4.1. CUS05 - Gestionar Producto 23
8.4.2. CUS06 - Gestionar Usuario 24
8.4.3. CUS07 - Gestionar Rol 24
8.4.4. CUS08 - Consultar Pedido 24
9. Gestión del Proyecto 25
9.1. Configuración Inicial y Organización del Proyecto 25
9.1.1. Creación del Repositorio 25
9.1.2. Estructura del proyecto 26
9.1.3. Creación de la rama develop 27
9.1.4. Creación de los permisos, y adición de los otros developers 28
9.1.5. Creación del tablero Kanban y asignación de tareas 29
9.2. Ejecución de tareas asignadas usando flujo de ramas feature 30
9.2.1. Cristopher Bayona – Desarrollo de módulos Producto, Tipo Producto y Cliente 30
9.2.2. Roosevelt Parejas – Desarrollo de módulos Pedido y Detalle Pedido 34

9.2.3. María Collana – Desarrollo de módulos Mesa y Estado de Mesa..... 42

9.3. Ejecución de tareas asignadas subiendo respectivos documentación 46

9.3.1. Roosevelt Parejas – Subir requisitos_01 46

9.3.2. Loana Muñoz – Subir informe_apf1 49

9.3.3. María Collana – Subir readme 53

10. Bibliografias: 57

Agradecimientos

Queremos expresar nuestro agradecimiento a todas las personas que, de una u otra manera, contribuyeron al desarrollo de este proyecto. A nuestro docente, por brindarnos los conocimientos y la guía necesaria durante el proceso de aprendizaje; y a nuestra institución, por proporcionarnos las herramientas y el entorno adecuado para nuestra formación profesional.

Asimismo, agradecemos el apoyo y la colaboración entre los integrantes del grupo, ya que el trabajo en equipo fue fundamental para lograr los objetivos planteados.

Dedicatoria

Dedicamos este trabajo a nosotros mismos, por el esfuerzo, la dedicación y la constancia que pusimos en cada etapa del desarrollo del proyecto. También lo dedicamos a nuestras familias, quienes han sido un pilar importante en nuestro camino académico, brindándonos apoyo y motivación para seguir adelante.

Finalmente, lo dedicamos a todas aquellas personas que buscan superarse día a día y alcanzar sus metas, recordando que el aprendizaje es un proceso continuo.

Universidad Tecnológica del Perú

Resumen

El presente proyecto consiste en el desarrollo de un sistema de gestión de restaurantes utilizando tecnologías modernas como Spring Boot para el backend, MySQL para la base de datos y GitHub como herramienta de control de versiones. Este sistema está orientado a optimizar los procesos operativos dentro de un restaurante, integrando módulos clave como la gestión de inventario (insumos y menú), administración de mesas en tiempo real, facturación, generación de reportes de ventas y gestión de usuarios según roles (meseros, administradores y cocineros).

La implementación de este sistema busca mejorar la eficiencia en la atención al cliente, reducir errores en la toma de pedidos y control de stock, y facilitar la toma de decisiones mediante reportes automatizados. Asimismo, se promueve el trabajo colaborativo mediante el uso de herramientas de desarrollo modernas, aplicando buenas prácticas en el manejo de versiones y desarrollo de software.

Universidad Tecnológica del Perú

Abstract

This project involves the development of a restaurant management system using modern technologies such as Spring Boot for the backend, MySQL for database management, and GitHub for version control. The system aims to optimize restaurant operations by integrating key modules such as inventory management (supplies and menu), real-time table management, billing system, sales reporting, and user management based on roles (waiters, administrators, and chefs).

The implementation of this system seeks to improve customer service efficiency, reduce errors in order handling and stock control, and support decision-making through automated reports. Additionally, it encourages collaborative work through the use of modern development tools and best practices in version control and software development.

1. Realidad Problematica

En la actualidad, muchos restaurantes presentan deficiencias en la gestión de sus procesos operativos, especialmente en el control de inventarios, lo que genera pérdidas económicas, desperdicio de insumos y una planificación ineficiente. La gestión del abastecimiento y de la cocina es un factor clave en este tipo de negocios, ya que una mala administración puede afectar directamente la rentabilidad y la satisfacción del cliente (Cruz Ulloa, 2024).

Asimismo, diversas investigaciones evidencian que la falta de sistemas formales de control de inventarios en restaurantes ocasiona problemas en la gestión financiera y limita la toma de decisiones. En muchos casos, las empresas no cuentan con políticas ni procesos definidos, lo que impide un adecuado control interno y repercute negativamente en su rentabilidad (Restrepo et al., 2020).

Además, el uso limitado de herramientas tecnológicas en restaurantes reduce su competitividad, ya que actualmente la digitalización permite mejorar procesos internos y brindar un mejor servicio al cliente. La implementación de sistemas informáticos en restaurantes se ha convertido en una necesidad para optimizar operaciones y generar ventajas competitivas en el mercado (Pinos Robles, 2021).

En este contexto, se evidencia la necesidad de desarrollar soluciones tecnológicas que integren la gestión de inventarios, pedidos, usuarios y reportes, con el fin de mejorar la eficiencia operativa y la calidad del servicio en los restaurantes.

2. Justificación del Proyecto

El desarrollo de un sistema de gestión de restaurantes se justifica por la necesidad de optimizar los procesos operativos y mejorar la eficiencia en la administración del negocio. La implementación de una solución tecnológica permitirá automatizar tareas como el control de inventario, la gestión de mesas, la facturación y la generación de reportes de ventas, reduciendo significativamente los errores humanos y mejorando la calidad del servicio.

Asimismo, este sistema facilitará la coordinación entre los distintos roles del restaurante (meseros, cocineros y administradores), permitiendo una comunicación más fluida y organizada. Esto se traduce en una mejor experiencia para el cliente y en una mayor productividad del personal.

Desde el punto de vista empresarial, contar con información en tiempo real permitirá tomar decisiones más acertadas, optimizar recursos y aumentar la rentabilidad del negocio (Oracle Hospitality, s.f.). Además, el uso de tecnologías como Spring Boot, MySQL y GitHub garantiza el desarrollo de una solución escalable, segura y alineada con las prácticas actuales de la ingeniería de software.

Finalmente, este proyecto también se justifica en el ámbito académico, ya que permite aplicar conocimientos teóricos en un caso práctico real, fortaleciendo habilidades en desarrollo backend, gestión de bases de datos y trabajo colaborativo mediante control de versiones, competencias esenciales para el desempeño profesional.

3. Objetivos

3.1. Objetivo General:

Desarrollar un sistema de gestión para restaurantes que contribuya a optimizar los

procesos operativos del negocio, a través de la implementación de funcionalidades como el control de inventarios, la gestión de pedidos, la administración de usuarios y la generación de reportes. Este sistema busca mejorar las actividades diarias del restaurante como reducir errores en los procesos manuales, facilitar el control de los recursos disponibles y apoyar la toma de decisiones mediante información actualizada. Con el fin de mejorar la calidad del servicio brindado al cliente y aportar a una mejor rentabilidad y productividad del negocio mediante el uso de la tecnología.

3.2. Objetivos Específicos:

- Analizar los procesos operativos de un restaurante para identificar las principales dificultades relacionadas con el control de inventario, la gestión de pedidos y la administración general, con el fin de plantear una solución adecuada a estas necesidades.
- Diseñar una base de datos estructurada que permita organizar y gestionar de manera eficiente la información relacionada con productos, usuarios, pedidos, ventas e inventario dentro del sistema.
- Desarrollar e implementar el backend del sistema utilizando Spring Boot para automatizar procesos como la gestión de pedidos, el control de inventarios y la administración de usuarios.
- Integrar una base de datos en MySQL que permita almacenar, consultar y administrar la información del sistema de manera segura y ordenada.
- Utilizar GitHub como herramienta de control de versiones para llevar un mejor seguimiento del desarrollo del proyecto y fomentar el trabajo colaborativo entre los integrantes del equipo.

4. Marco teórico:

4.1. Sistemas de Gestión de Restaurantes

Un sistema de gestión de restaurantes es una solución tecnológica diseñada para apoyar y optimizar las operaciones que se realizan dentro de un restaurante, permitiendo administrar procesos como pedidos, control de inventarios, ventas, usuarios y generación de reportes. Este tipo de sistemas buscan automatizar tareas que normalmente se realizan de manera manual, reduciendo errores, mejorando los tiempos de respuesta y brindando mayor organización en la administración del negocio.

Además, la implementación de estos sistemas permite mejorar la eficiencia operativa y ofrecer un mejor servicio al cliente, ya que facilita el control de las actividades diarias y contribuye a una mejor toma de decisiones.

4.2. Gestión de inventarios

La gestión de inventarios es uno de los procesos más importantes dentro de un restaurante, ya que permite controlar el ingreso, almacenamiento y salida de insumos o productos necesarios para la operación. Un adecuado manejo del inventario ayuda a evitar pérdidas económicas, desperdicios, faltantes de stock y problemas en la atención al cliente.

También permite optimizar recursos, planificar compras de manera más eficiente y mantener un mejor control sobre los costos del negocio, siendo un factor importante para la rentabilidad del restaurante.

4.3. Automatización de procesos

La automatización de procesos consiste en el uso de herramientas tecnológicas para ejecutar tareas de manera más rápida, ordenada y eficiente, reduciendo la intervención manual. En el contexto de los restaurantes, permite agilizar actividades como registrar pedidos, controlar inventarios, emitir reportes y administrar información.

Su aplicación contribuye a disminuir errores humanos, mejorar la productividad del personal y optimizar el funcionamiento general del negocio.

4.4. Sistemas de información

Los sistemas de información son conjuntos de herramientas, procesos y recursos tecnológicos que permiten recopilar, almacenar, procesar y administrar datos para apoyar las operaciones de una organización y la toma de decisiones.

En un restaurante, estos sistemas facilitan el manejo de información relacionada con ventas, inventarios, usuarios y pedidos, permitiendo tener datos organizados, actualizados y disponibles para una mejor gestión.

5. Tecnologías utilizadas en el desarrollo

Para el desarrollo del sistema se emplearán herramientas y tecnologías que permitan construir una solución robusta y escalable.

5.1. Spring boot

Es un framework que facilita el desarrollo de aplicaciones backend, permitiendo construir servicios eficientes, seguros y escalables mediante una arquitectura organizada.

5.2. MySQL

Es un sistema gestor de bases de datos relacional que permite almacenar, organizar y administrar la información del sistema de forma segura y estructurada.

5.3. GitHub

Es una plataforma que permite gestionar versiones del código fuente, facilitar el trabajo colaborativo y mantener control sobre los cambios realizados durante el desarrollo del proyecto.

5.4. Ingeniería de software

La ingeniería de software proporciona principios, métodos y buenas prácticas para el análisis, diseño, desarrollo y mantenimiento de sistemas informáticos. Su aplicación en este proyecto permite seguir una estructura organizada para el desarrollo del sistema, asegurando calidad, escalabilidad y cumplimiento de los requerimientos establecidos.

Además, involucra procesos como levantamiento de requisitos, diseño del sistema, desarrollo, pruebas e implementación, fundamentales para la construcción de una solución tecnológica funcional.

6. Metodología

6.1. Tipo de investigación

En este proyecto se presenta bajo una investigación, ya que se busca dar solución ante una problemática real identificada en la gestión operativa de los restaurantes, a través de un desarrollo de sistema que permita optimizar los procesos como el control de inventarios, gestión de pedidos y administración del negocio.

6.2. Enfoque de investigación

Este proyecto tiene un enfoque cuantitativo, debido a que busca evaluar las mejoras en los procesos operativos mediante indicaciones como reducción de errores, tiempos de atención, control de recursos y eficiencia en la gestión.

6.3. Diseño de investigación

El diseño de investigación es no experimental, ya que este analiza la problemática existente sin manipular variables, proponiendo una solución tecnológica basada en las necesidades vistas.

6.4. Metodología de desarrollo

Para este desarrollo el sistema se empleará la metodología Scrum, esto se debe a que permitirá organizar el trabajo de manera iterativa, facilitando el proceso por etapas, la adaptación a cambios y el seguimiento del proyecto.

- Las etapas consideradas son:
- Levantamiento de requerimientos
- Diseño del sistema y base de datos
- Desarrollo e implementación
- Pruebas del sistema

- Implementación y evaluación

6.5. Herramientas tecnológicas

- Las herramientas que se utilizaran son las siguientes:
- Java
- Spring Boot
- MySQL
- GitHub
- Postman para pruebas de APIs

7. Requisitos

7.1. Requisitos Funcionales

- El sistema debe permitir gestionar pedidos (crear, editar, eliminar).
- El sistema debe permitir gestionar mesas.
- El sistema debe permitir administrar productos.
- El sistema debe permitir gestionar usuarios.
- El sistema debe permitir gestionar roles.
- El sistema debe permitir asociar pedidos a mesas y usuarios.
- El sistema debe permitir consultar pedidos.

7.2. Requisitos No Funcionales

- El sistema debe ser eficiente en el procesamiento de pedidos.
- El sistema debe garantizar disponibilidad durante su uso.
- El sistema debe ser fácil de usar.
- El sistema debe ser escalable para futuras mejoras.
- El sistema debe asegurar la integridad de los datos.

7.3. Restricciones Técnicas

- El sistema debe estar desarrollado utilizando Spring Boot.
- El sistema debe exponer servicios mediante arquitectura REST.
- El sistema debe utilizar JPA/Hibernate para la persistencia de datos.
- El sistema debe emplear una base de datos relacional.
- El sistema debe permitir la realización de pruebas básicas bajo el enfoque TDD.

7.4. Alcance del Sistema

Funcionalidades No Incluidas

- No se incluye el desarrollo de una interfaz gráfica (frontend).
- No se incluye la implementación de autenticación y autorización avanzada (JWT, Spring Security).
- No se incluye lógica de negocio compleja como optimización de pedidos o control automático de inventario.
- No se incluye la generación de reportes o dashboards.
- No se incluyen integraciones externas ni notificaciones.

8. Diagramas

8.1. Diagrama Entidad-Relación (ER)

El presente diagrama entidad-relación representa la estructura de la base de datos del sistema de gestión de restaurante. En él se identifican las principales entidades, sus atributos y las relaciones que existen entre ellas, permitiendo modelar el funcionamiento del sistema de manera organizada.

En primer lugar, se observa la entidad Cliente, la cual almacena la información básica de los clientes, como su DNI y nombre. Esta entidad se relaciona con Pedido, ya que un cliente puede realizar uno o varios pedidos, estableciendo una relación de uno a muchos.

La entidad Pedido es central en el sistema, ya que registra la información de cada pedido realizado, incluyendo su estado, fecha y total. Además, se vincula con varias entidades: con Cliente, con Mesa (indicando en qué mesa se realizó el pedido) y con Usuario, que representa al mesero o empleado que atendió dicho pedido.

Por otro lado, la entidad Detalle_Pedido permite descomponer cada pedido en los productos específicos que lo conforman. Aquí se registra la cantidad y el subtotal de cada producto dentro del pedido. Esta entidad tiene una relación de muchos a uno tanto con Pedido como con Producto, ya que un pedido puede tener varios detalles y cada detalle corresponde a un producto específico.

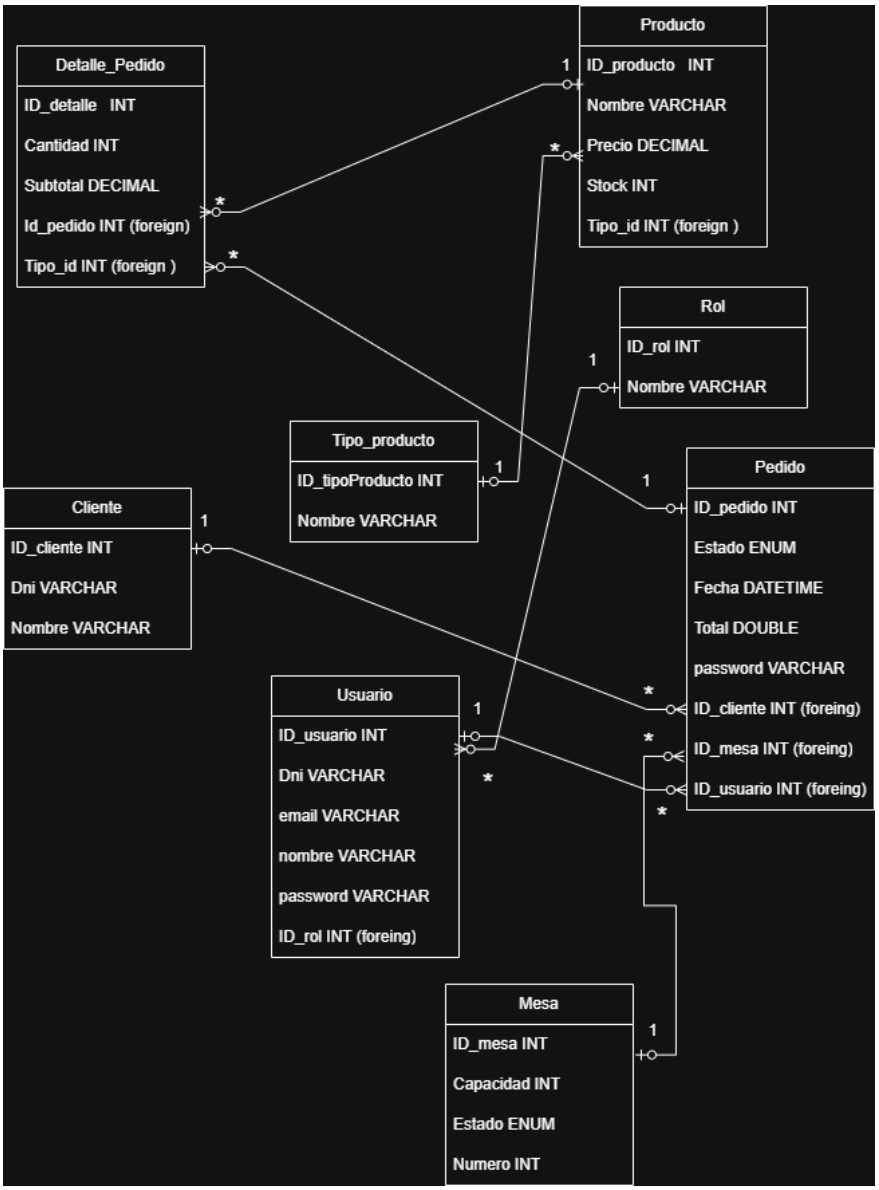
La entidad Producto contiene la información de los productos disponibles en el restaurante, como nombre, precio y stock. Además, cada producto pertenece a un Tipo_producto, lo que permite clasificar los productos (por ejemplo, bebidas, platos, postres), estableciendo una relación de muchos a uno.

En cuanto a la gestión del personal, la entidad Usuario almacena los datos de los empleados del sistema, incluyendo credenciales de acceso. Cada usuario está asociado a un

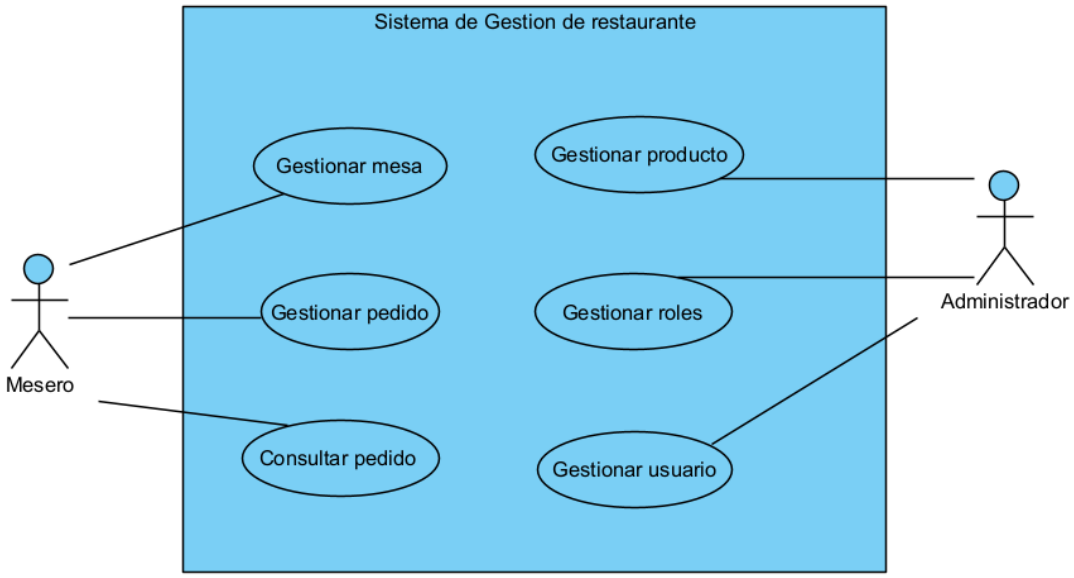
Rol, lo que define sus permisos dentro del sistema (por ejemplo, administrador o mesero), mediante una relación de muchos a uno.

Finalmente, la entidad Mesa representa las mesas del restaurante, incluyendo su capacidad, estado y número. Esta se relaciona con Pedido, ya que una mesa puede tener múltiples pedidos a lo largo del tiempo.

En conjunto, este modelo permite gestionar de manera eficiente la información del restaurante, facilitando el control de pedidos, productos, clientes y usuarios, así como las relaciones entre ellos.



8.2. Diagrama de Casos de Uso



8.3. Actor: Mesero

8.3.1. CUS01 - Registrar Pedido

Descripción:

El mesero registra un nuevo pedido en el sistema, asociándolo a una mesa y a los productos solicitados.

Flujo básico:

- El mesero ingresa al sistema.
- Selecciona la opción registrar pedido.
- Ingresa los datos del pedido (mesa, productos, cantidades).
- El sistema guarda la información del pedido.

8.3.2. CUS02 - Gestionar Mesa

Descripción:

El mesero actualiza el estado de una mesa (libre u ocupada).

Flujo básico:

- El mesero selecciona una mesa.
- Modifica su estado.
- El sistema actualiza la información.

8.3.3. CUS03 - Gestionar Pedido

Descripción:

El mesero puede actualizar información de un pedido, como modificar productos o cambiar su estado.

Flujo básico:

- El mesero selecciona un pedido existente.
- Realiza modificaciones.
- El sistema guarda los cambios.

8.3.4. CUS04 - Consultar Pedido

Descripción:

El mesero visualiza los pedidos registrados en el sistema.

Flujo básico:

- El mesero accede a la lista de pedidos.
- El sistema muestra la información disponible.

8.4. Actor: Administrador

8.4.1. CUS05 - Gestionar Producto

Descripción:

El administrador crea, actualiza o elimina productos del sistema.

Flujo básico:

- El administrador accede al módulo de productos.
- Realiza la acción (crear, editar o eliminar).
- El sistema guarda los cambios.

8.4.2. CUS06 - Gestionar Usuario

Descripción:

El administrador administra los usuarios del sistema.

Flujo básico:

- El administrador accede al módulo de usuarios.
- Crea, edita o elimina usuarios.
- El sistema guarda la información.

8.4.3. CUS07 - Gestionar Rol

Descripción:

El administrador asigna y modifica roles de usuario.

Flujo básico:

- El administrador accede al módulo de roles.
- Realiza modificaciones.
- El sistema actualiza los datos.

8.4.4. CUS08 - Consultar Pedido

Descripción:

El administrador visualiza los pedidos registrados en el sistema.

Flujo básico:

- El administrador accede a la lista de pedidos.
- El sistema muestra la información.

9. Gestión del Proyecto

9.1. Configuración Inicial y Organización del Proyecto

9.1.1. Creación del Repositorio

Se realiza la creación del nuevo repositorio con su respectivo nombre que en este caso es “Gestion_De_Restaurante” dicho repositorio va a permitir que logremos trabajar en un lugar centralizado, para poder registrar las tareas, avances además de poder asignar trabajos a los demás developers

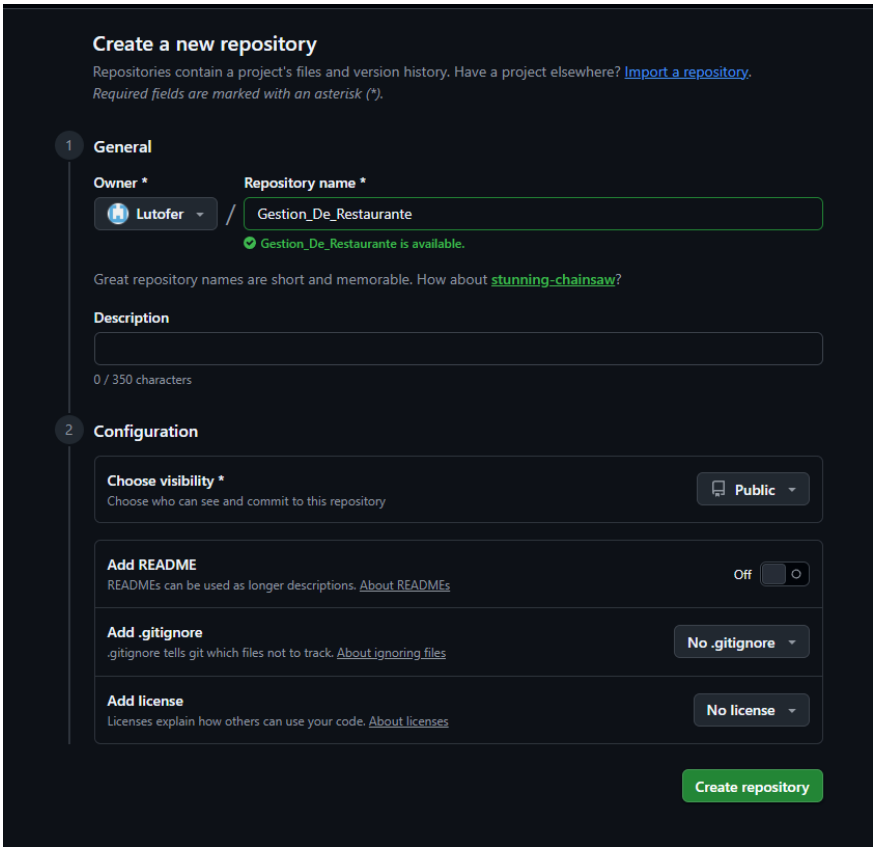


Figura 3. Creación del repositorio en GitHub

(Fuente: Elaboración propia)

9.1.2. Estructura del proyecto

Replicamos la estructura de las carpetas, las cuales se especifica en el modelo, además dicha estructura nos permite separar el código, el readme, los informes, resultados, docs, entre otras cosas. Adicionalmente todo lo agregamos a la rama principal.

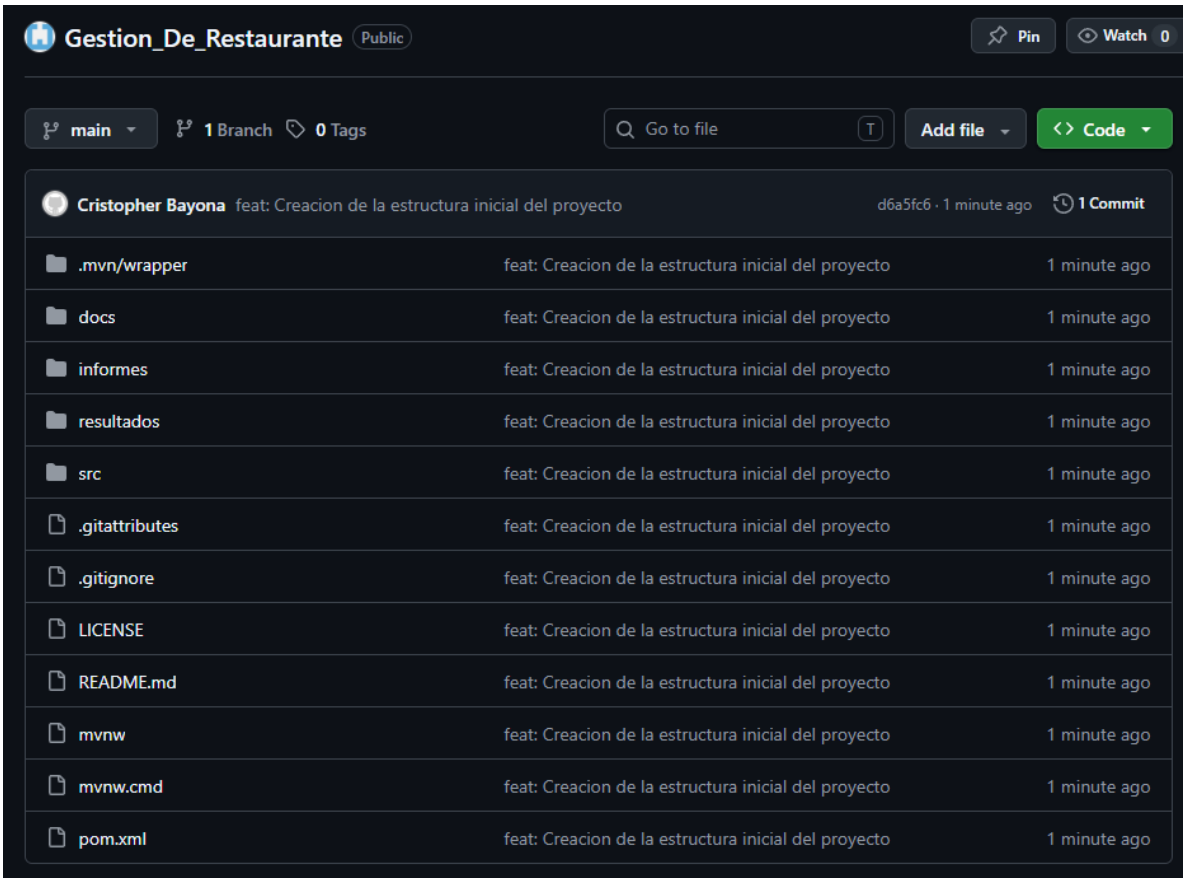


Figura 4. Creación de la estructura del proyecto en GitHub

(Fuente: Elaboración propia)

Comandos utilizados:

- Git init
- Git status
- Git add .

- git commit –m “feat: Creacion de la estructura inicial del proyecto”
- git branch -M main
- git remote add origin https://github.com/Lutofer/Gestion_De_Restaurante
- git push -u origin main

9.1.3. Creación de la rama develop

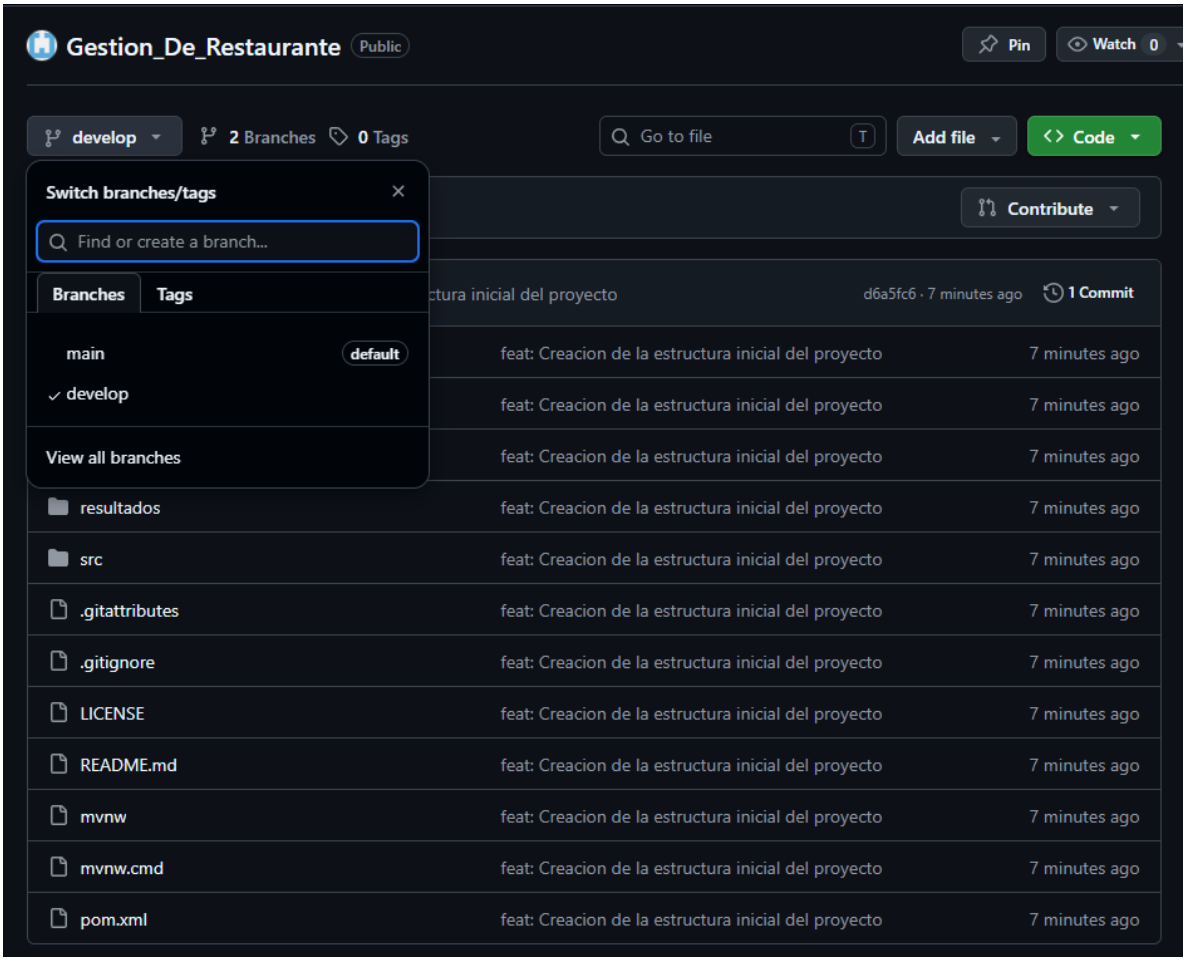


Figura 5. Creación de la rama develop

(Fuente: Elaboración propia)

Comandos utilizados:

- git checkout -b develop
- git push -u origin develop

9.1.4. Creación de los permisos, y adición de los otros developers

En lo que respecta a los permisos de la rama main y develop, se le agrega una regla para que se deba tener como mínimo una aprobación de un reviewer para que se pueda hacer un merge, luego se le envía la invitación a nuestros otros compañeros para que puedan trabajar en conjunto

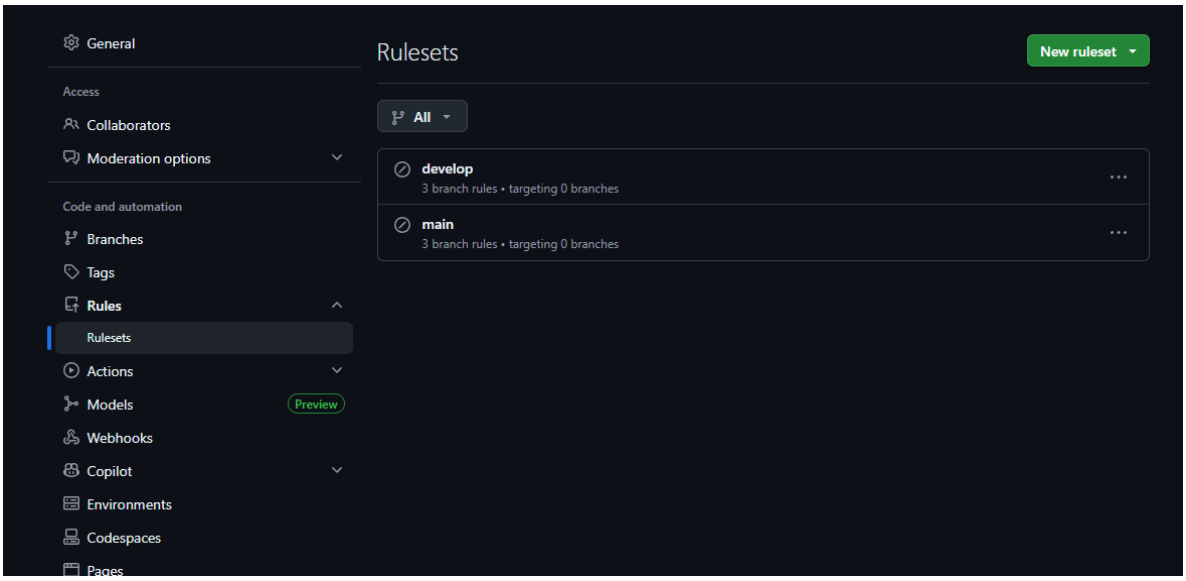


Figura 6. Creación de las reglas de la rama main y develop

(Fuente: Elaboración propia)

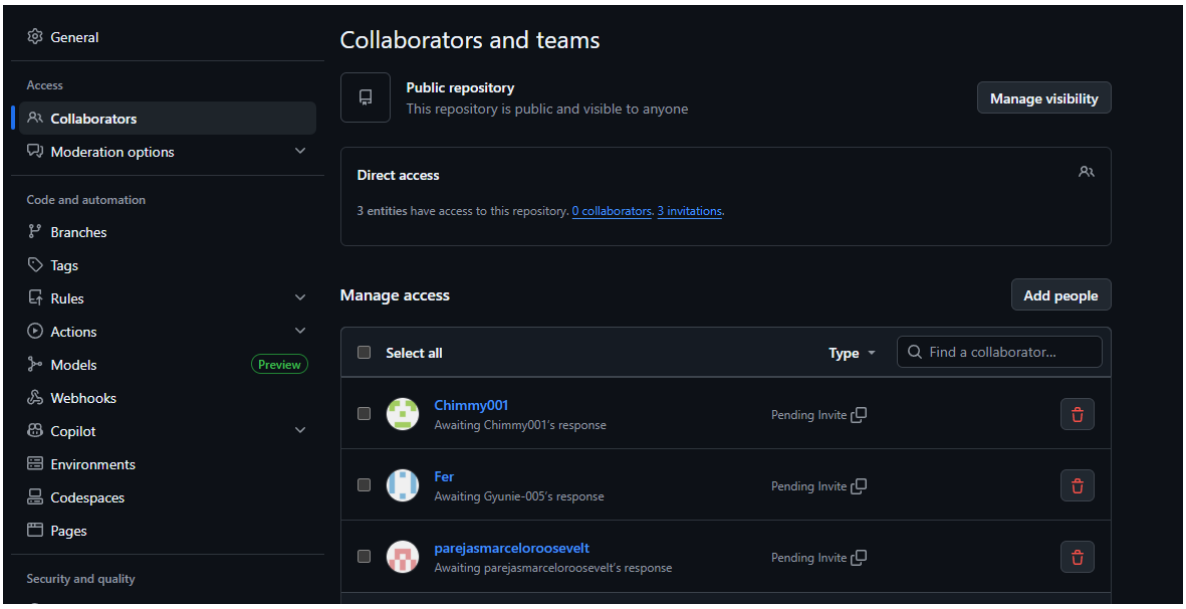


Figura 7. Invitación de los Collaborators

(Fuente: Elaboración propia)

9.1.5. Creación del tablero Kanban y asignación de tareas

Al realizar la creación del tablero Kanban se pueden empezar a crear Issues y asignar respectivamente a cada develop, para que estos puedan ir avanzando en su trabajo, adicionalmente se dejan varios Issues en backlog más que nada porque son Issues que se van a completar en el futuro. De esta manera debería quedar la tabla después de haber trabajado

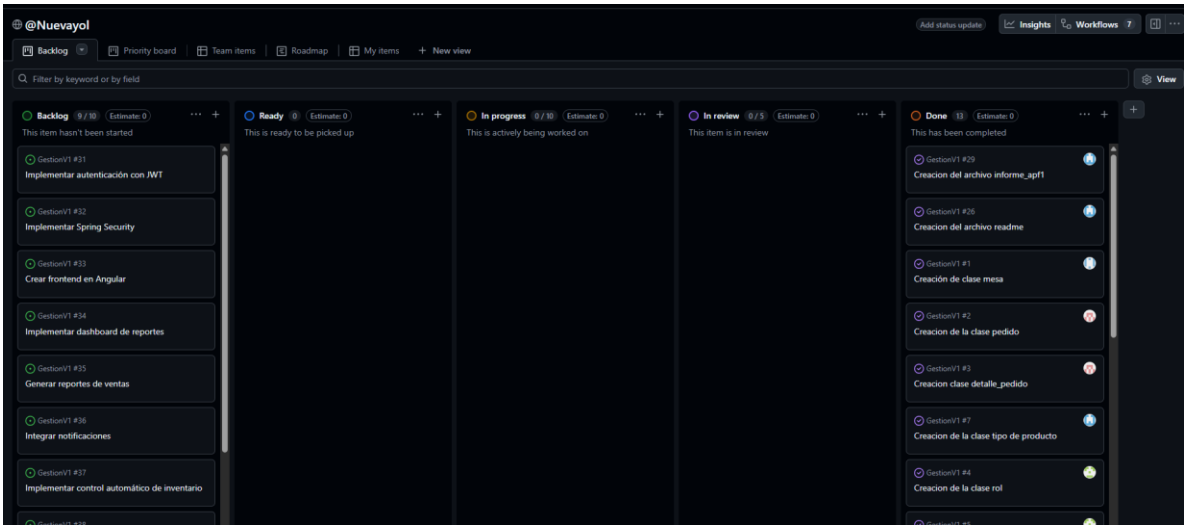
todos.

Figura 8. Tabla Kanban del proyecto de Github

(Fuente: Elaboración propia)

9.2. Ejecución de tareas asignadas usando flujo de ramas feature

A partir de la asignación de tareas, cada integrante del equipo desarrolló sus funcionalidades mediante el uso de ramas feature/*, permitiendo trabajar de manera aislada y



organizada sobre cada requerimiento definido en las issues. Este enfoque facilita el control de versiones, la integración progresiva y la revisión del código antes de su incorporación a la rama develop.

9.2.1. Cristopher Bayona – Desarrollo de módulos Producto, Tipo Producto y Cliente

Ramas utilizadas: feature/creacion-producto-crud, feature/creacion-tipo-producto-crud, feature/creacion-cliente-crud

Descripción del proceso:

Paso 1: Creación de la rama main

Se crea la rama correspondiente para el desarrollo del módulo producto a partir de la rama

develop.

Comando utilizado:

git checkout -b feature/creacion-producto-crud

```
Nosee@Nose MINGW64 ~/Desktop/testborrar/Gestuib/Gestion_De_Restaurante (main)
$ git checkout -b feature/creacion-producto-crud
Switched to a new branch 'feature/creacion-producto-crud'

Nosee@Nose MINGW64 ~/Desktop/testborrar/Gestuib/Gestion_De_Restaurante (feature/
creacion-producto-crud)
```

Figura 9. Creación de la rama feature/creacion-producto-crud

(Fuente: Elaboración propia)

Paso 2: Desarrollo de la funcionalidad

Se implementan las capas entity, repository, service y controller para el módulo producto, siguiendo la arquitectura del proyecto.

```
EXPLORER
GESTION_DE_RESTAURANTE
  .mvn
  .vscode
  docs
  informes
  resultados
  src
    main
      java \ com \ Eugeo \ gestion_restaura...
        Controller
          .gitkeep
          ProductoController.java                U
        Entity
          .gitkeep
          Producto.java                          9, U
        Repository
          .gitkeep
          ProductoRepository.java                2, U
        Service
          .gitkeep
          ProductoService.java                   1, U
      GestionRestauranteApplication.java
      resources
    test
  target
  .gitattributes
  .gitignore
  LICENSE
  mvnw
  mvnw.cmd
  pom.xml                                        1
  README.md
```

```
.gitkeep   Producto.java 9, U   ProductoRepository.java 2, U   ProductoController.java U   ProductoService.java 1, U
src > main > java > com > Eugeo > gestion_restaurante > Repository > ProductoRepository.java > Language Support for Java(TM) by Red Hat > ProductoRepository
1   package com.Eugeo.gestion_restaurante.Repository;
2
3   import java.util.List;
4
5   import org.springframework.data.jpa.repository.JpaRepository;
6   import org.springframework.stereotype.Repository;
7
8   import com.Eugeo.gestion_restaurante.Entity.Producto;
9   import com.Eugeo.gestion_restaurante.Entity.TipoProducto;
10
11  @Repository
12  public interface ProductoRepository extends JpaRepository<Producto, Long> {
13
        Show AOT-generated Implementation, Query, etc...
14      List<Producto> findByNombreContainingIgnoreCase(String nombre);
15
        Show AOT-generated Implementation, Query, etc...
16      List<Producto> findByTipo(TipoProducto tipo);
17
        Show AOT-generated Implementation, Query, etc...
18      List<Producto> findByStockGreaterThan(Integer stock);
19  }
```

Figura 10. Implementación de los módulos de Producto

(Fuente: Elaboración propia)

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

git add .

git commit -m "feat: implementa CRUD de Producto (entity, repository, service, controller) (Closes #6)"

git push -u origin feature/creacion-producto-crud

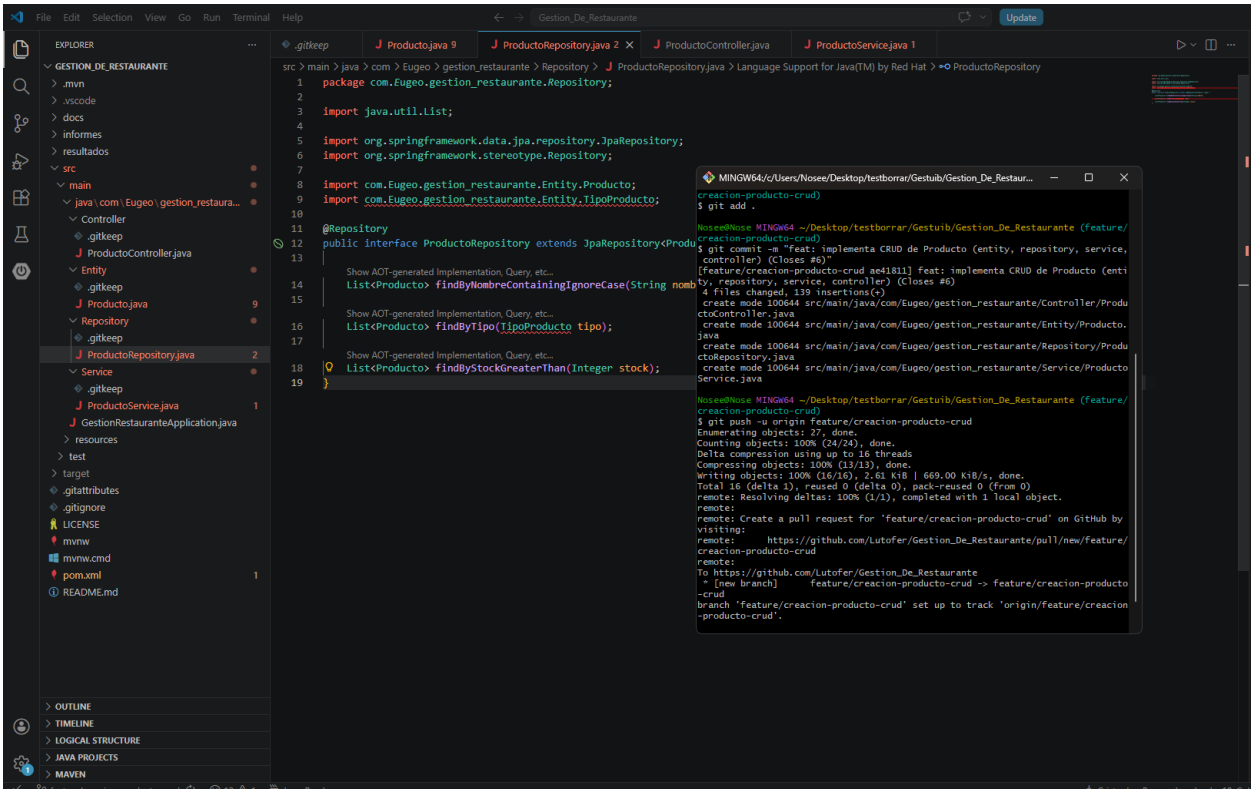


Figura 11. Registro de cambios en el repositorio

(Fuente: Elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

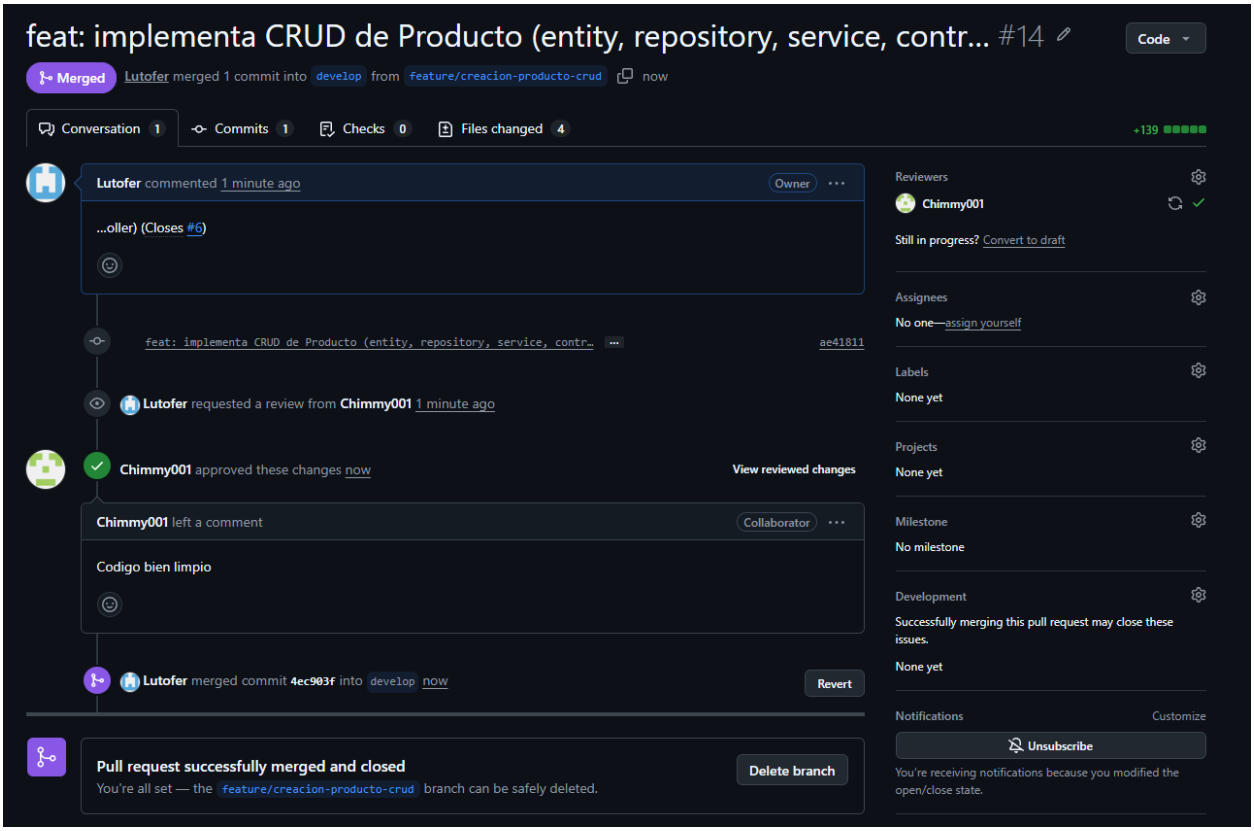


Figura 12. Merge de la rama feature hacia develop

(Fuente: Elaboración propia)

De igual manera se realiza con cada rama que se le asignó a Cristopher de esta manera completa sus 3 Issues asignados en el tablero Kanban

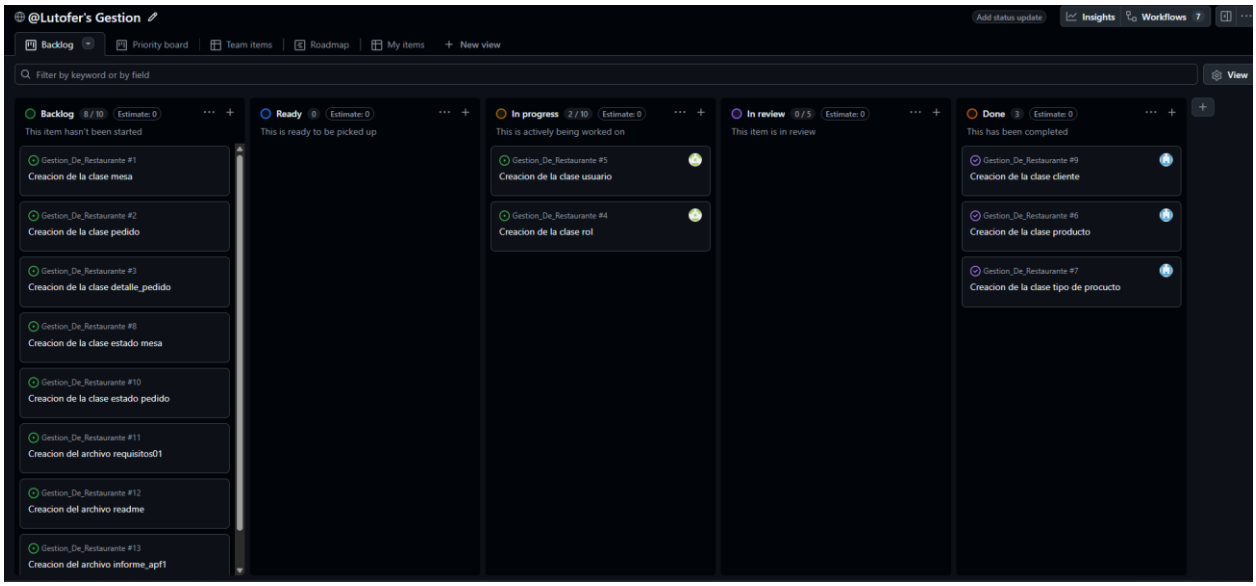


Figura 13. Tareas completadas en el tablero Kanban

(Fuente: Elaboración propia)

9.2.2. Roosevelt Parejas – Desarrollo de módulos Pedido y Detalle Pedido

Ramas utilizadas: feature/creacion-pedido-crud, feature/creacion-detalle-pedido-crud, feature/creacion-estado-pedido,

Descripción del proceso:

Paso 1: Creación de la rama feature

Se crea la rama correspondiente para el desarrollo del módulo producto a partir de la rama develop.

Comando utilizado:

git checkout -b feature/creacion-pedido-crud

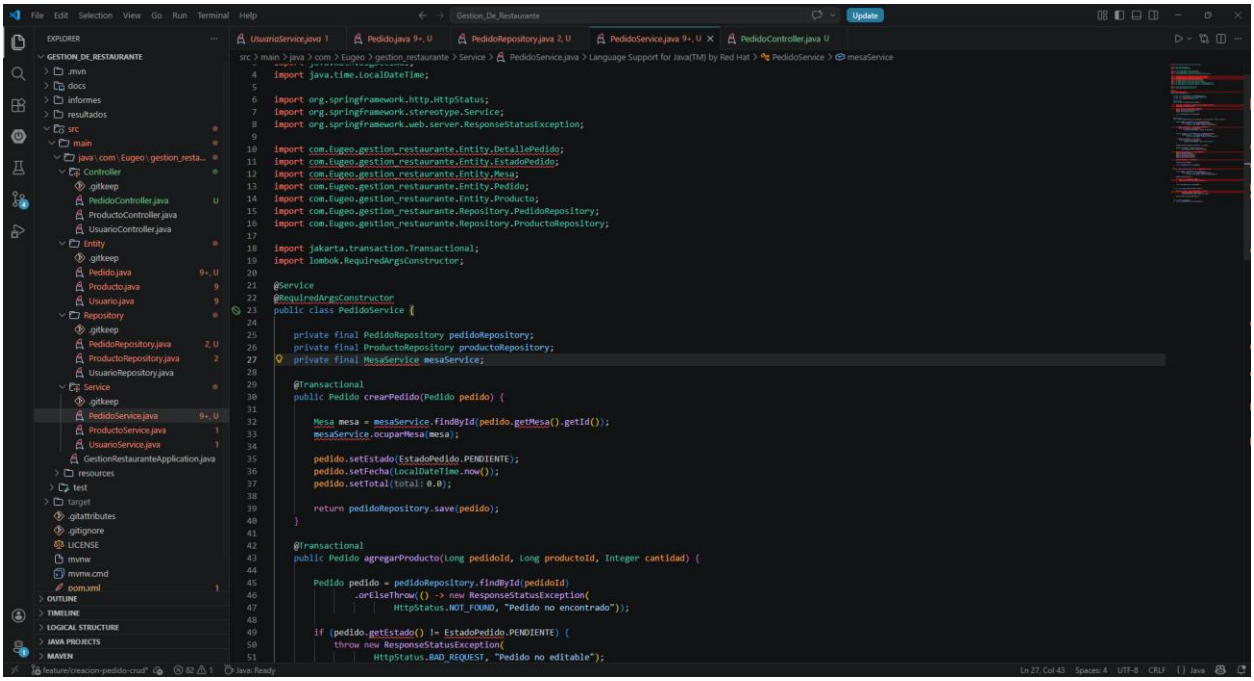
```
EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (main)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (develop)
$ git checkout -b feature/creacion-pedido-crud
Switched to a new branch 'feature/creacion-pedido-crud'

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-ped
ido-crud)
$ |
```

Figura 14. Creación de la rama feature/creacion-pedido-crud

(Fuente: Elaboración propia)

Paso 2: Desarrollo de las funcionalidades

Se pasa a implementar las capas de Entity, Repository, Service y Controller de la clase Pedido, siguiendo con el modelo del proyecto MVC.



```
import java.time.LocalDateTime;

import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

import com.Eugeo.gestion_restaurante.Entity.DetallePedido;
import com.Eugeo.gestion_restaurante.Entity.EstadoPedido;
import com.Eugeo.gestion_restaurante.Entity.Mesa;
import com.Eugeo.gestion_restaurante.Entity.Pedido;
import com.Eugeo.gestion_restaurante.Entity.Producto;
import com.Eugeo.gestion_restaurante.Repository.PedidoRepository;
import com.Eugeo.gestion_restaurante.Repository.ProductoRepository;

import jakarta.transaction.Transactional;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class PedidoService {

    private final PedidoRepository pedidoRepository;
    private final ProductoRepository productoRepository;
    private final MesaService mesaService;

    @Transactional
    public Pedido crearPedido(Pedido pedido) {

        Mesa mesa = mesaService.findById(pedido.getMesa().getId());
        mesaService.ocuparMesa(mesa);

        pedido.setEstado(EstadoPedido.PENDIENTE);
        pedido.setFecha(LocalDateTime.now());
        pedido.setTotal(total: 0.0);

        return pedidoRepository.save(pedido);
    }

    @Transactional
    public Pedido agregarProducto(Long pedidoId, Long productoId, Integer cantidad) {

        Pedido pedido = pedidoRepository.findById(pedidoId)
                .orElseThrow(() -> new ResponseStatusException(
                        HttpStatus.NOT_FOUND, "Pedido no encontrado"));

        if (pedido.getEstado() != EstadoPedido.PENDIENTE) {
            throw new ResponseStatusException(
                    HttpStatus.BAD_REQUEST, "Pedido no editable");
```

Figura 15: Implementación de los módulos de la entidad pedido.

(Fuente: elaboración propia)

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

git add .

git commit -m "feat: implementacion CRUD de Pedido (entity, repository, service, controller) (Closes #2)"

git push -u origin feature/creacion-pedido-crud

```
EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (develop)
$ git checkout feature/creacion-pedido-crud
Already on 'feature/creacion-pedido-crud'

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-ped
ido-crud)
$ git add .

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-ped
ido-crud)
$ git commit -m "feat: implementacion CRUD de Pedido (entity, repository, servic
e, controller)(Closes #2)"
[feature/creacion-pedido-crud 01ef5f4] feat: implementacion CRUD de Pedido (enti
ty, repository, service, controller)(Closes #2)
 4 files changed, 211 insertions(+)
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Controller/Pedid
oController.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Entity/Pedido.ja
va
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Repository/Pedid
oRepository.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Service/PedidoSe
rvice.java

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-ped
ido-crud)
$ git push -u origin feature/creacion-pedido-crud
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 16 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (16/16), 3.48 KiB | 1.74 MiB/s, done.
Total 16 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/creacion-pedido-crud' on GitHub by vi
siting:
remote:      https://github.com/Lutofer/Gestion_De_Restaurante/pull/new/feature/
creacion-pedido-crud
remote:
To https://github.com/Lutofer/Gestion_De_Restaurante.git
 * [new branch]      feature/creacion-pedido-crud -> feature/creacion-pedido-cru
d
branch 'feature/creacion-pedido-crud' set up to track 'origin/feature/creacion-p
edido-crud'.
```

Figura 16. Registro de los cambios en el repositorio

(Fuente: elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

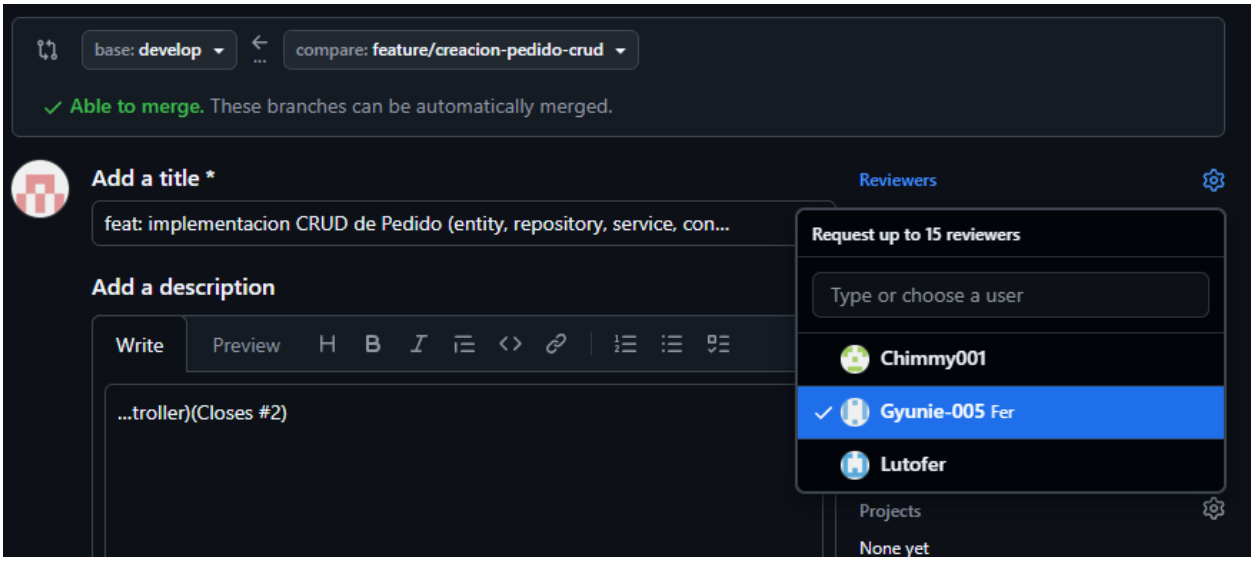


Figura 17. Asignacion del reviewer

(Fuente: Elaboración propia)

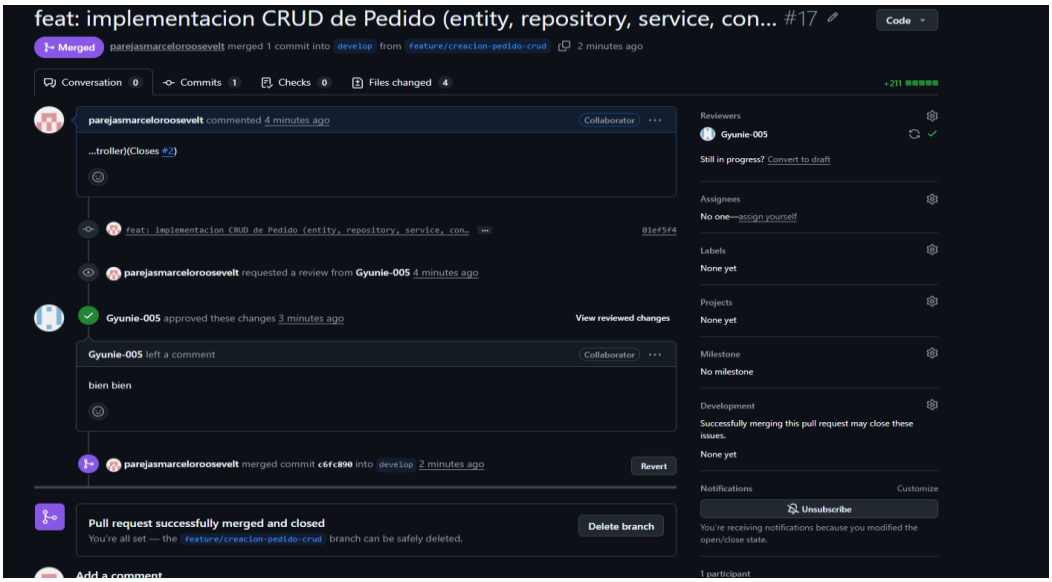


Figura 18. Merge desde la feature creada hacia la develop

(Fuente: Elaboración propia)

De igual manera se le realiza el mismo proceso a las dos siguientes Issues asignadas, lo que se representa en el tablero Kanban.

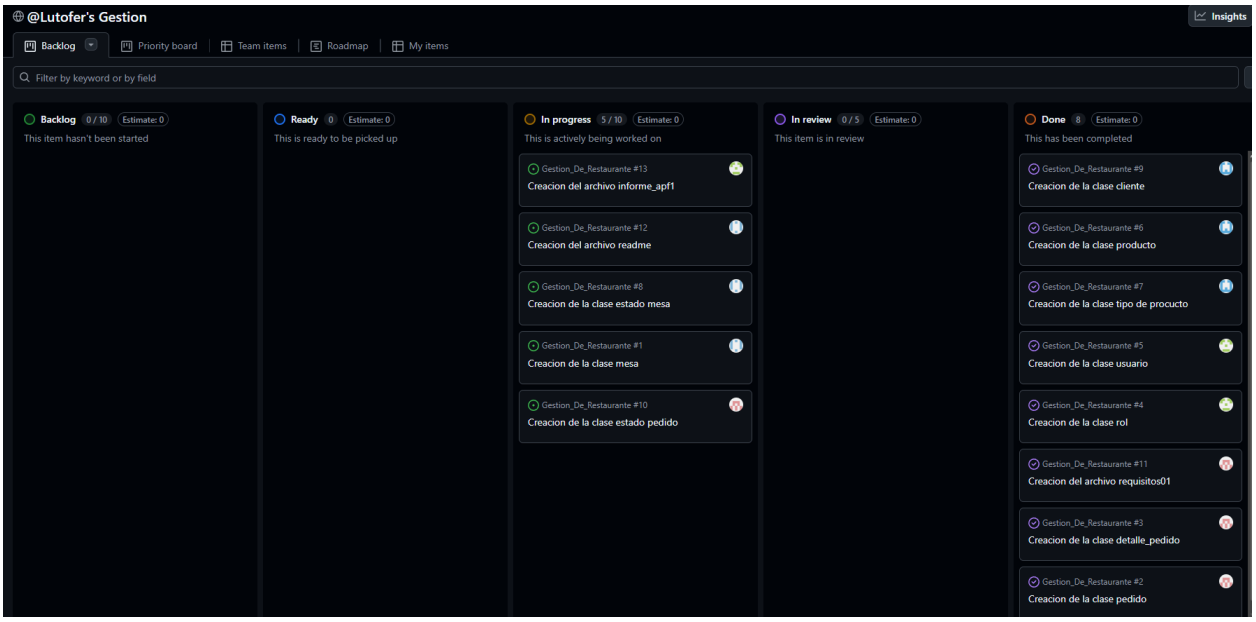


Figura 19. Tareas completadas en el tablero Kanban

(Fuente: Elaboración propia)

Loana Muñoz – Desarrollo de módulos Usuario y Rol

Ramas utilizadas: feature/creacion-usuario-crud, feature/creacion-tipo-rol-crud

Descripción del proceso:

Paso 1: Creación de la rama feature

Se crea la rama correspondiente para el desarrollo del módulo producto a partir de la rama main.

Comando utilizado:

git checkout -b feature/creacion-usuario-crud

```
ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc (master)
$ git checkout -b feature/creacion-usuario-crud
Switched to a new branch 'feature/creacion-usuario-crud'

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc (feature/creacion-usu
ario-crud)
$
```

Figura 20. Creación de la rama feature/creacion-usuario-crud

(Fuente: Elaboración propia)

Paso 2: Desarrollo de la funcionalidad

Se implementan las capas entity, repository, service y controller para el módulo producto, siguiendo la arquitectura del proyecto.

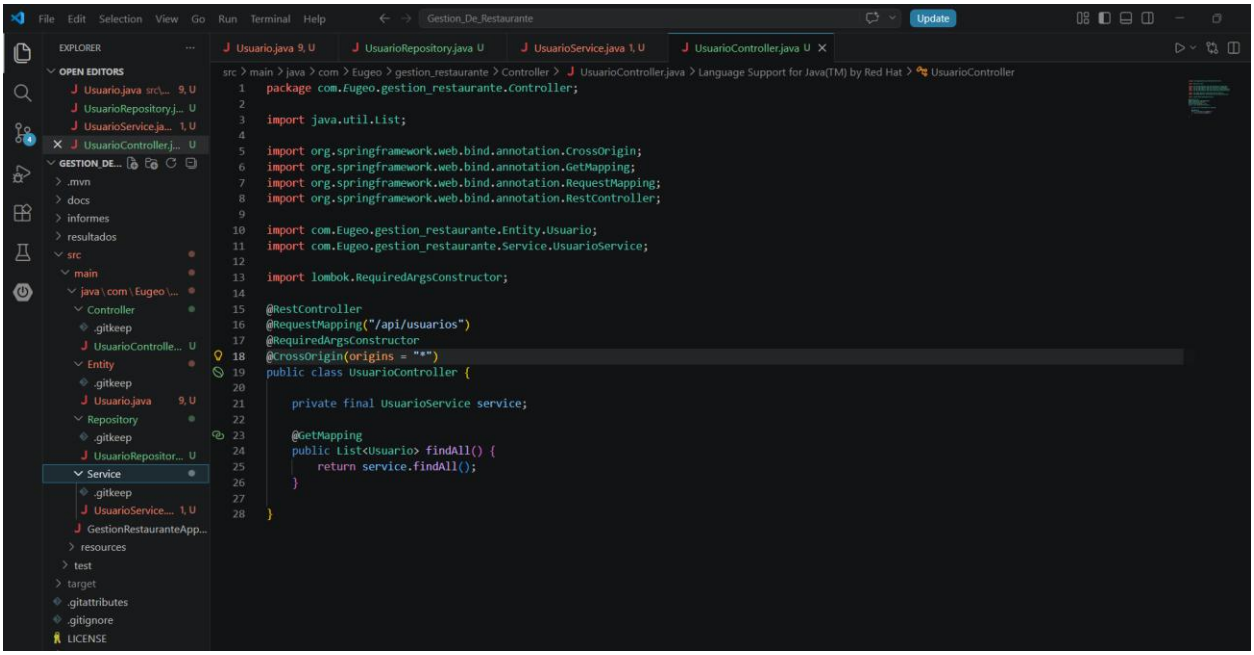


Figura 21. Implementación de los módulos de Usuario

(Fuente: Elaboración propia)

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

git add .

git commit -m "feat: implementa CRUD de Usuario (entity, repository, service, controller) (Closes #5)"

git push -u origin feature/creacion-usuario-crud

```
ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-usuario-crud)
$ git add .

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-usuario-crud)
$ git commit -m "feat: implementa CRUD de Usuario (entity, repository, service,
controller) (Closes #5)"
[feature/creacion-usuario-crud 204dae9] feat: implementa CRUD de Usuario (entity
, repository, service, controller) (Closes #5)
 4 files changed, 142 insertions(+)
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Controller/Usuar
ioController.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Entity/Usuario.j
ava
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Repository/Usuar
ioRepository.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Service/UsuarioS
ervice.java

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-usuario-crud)
$ git push -u origin feature/creacion-usuario-crud
fatal: unable to access 'https://github.com/Lutofer/Gestion_De_Restaurante.git/'
: schannel: next InitializeSecurityContext failed: SEC_E_ILLEGAL_MESSAGE (0x8009
0326) - This error usually occurs when a fatal SSL/TLS alert is received (e.g. h
andshake failed). More detail may be available in the Windows System event log.

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-usuario-crud)
$ git push -u origin feature/creacion-usuario-crud
Enumerating objects: 27, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 20 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (16/16), 2.56 KiB | 655.00 KiB/s, done.
Total 16 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/creacion-usuario-crud' on GitHub by v
isiting:
remote:      https://github.com/Lutofer/Gestion_De_Restaurante/pull/new/feature/
creacion-usuario-crud
remote:
To https://github.com/Lutofer/Gestion_De_Restaurante.git
 * [new branch]      feature/creacion-usuario-crud -> feature/creacion-usuario-c
rud
branch 'feature/creacion-usuario-crud' set up to track 'origin/feature/creacion-
usuario-crud'.

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-usuario-crud)
$ |
```

Figura 22. Registro de cambios en el repositorio

(Fuente: Elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

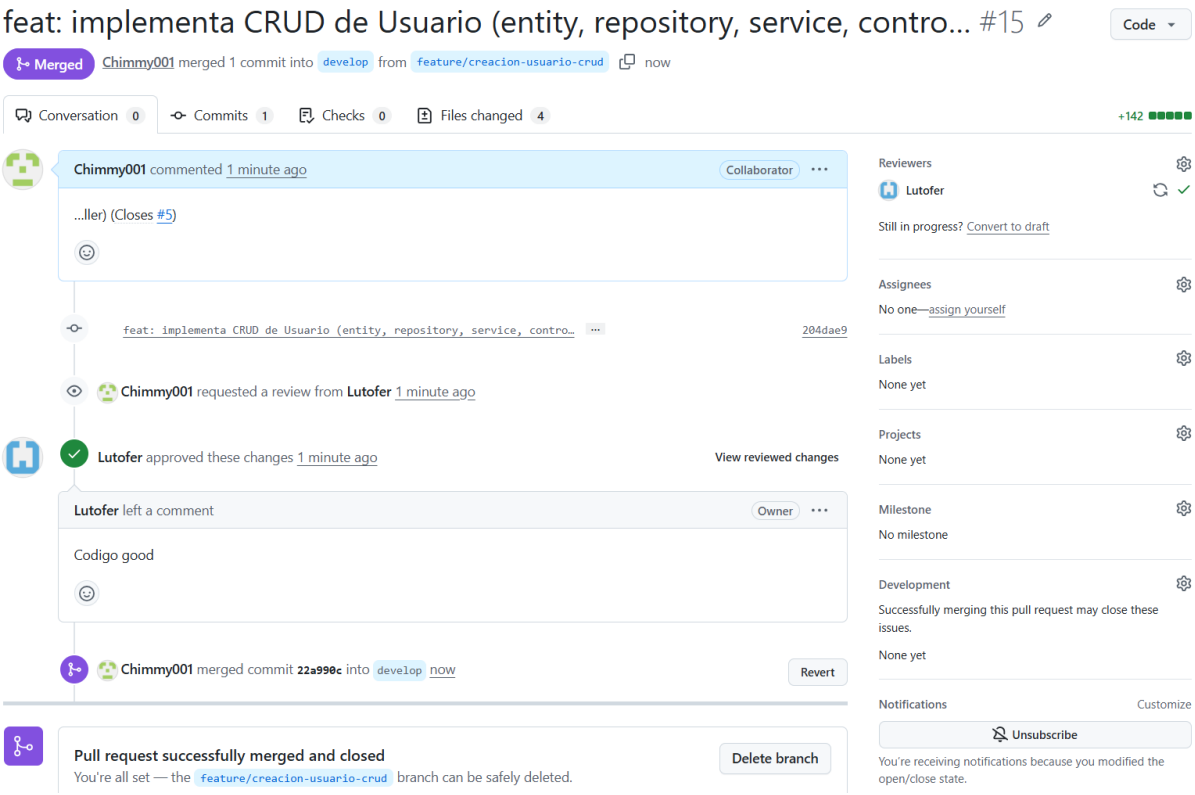


Figura 23. Merge de la rama feature hacia develop

(Fuente: Elaboración propia)

De igual manera se realiza con cada rama que se le asignó a loana de esta manera

completa sus 2 Issues asignados en el tablero Kanban

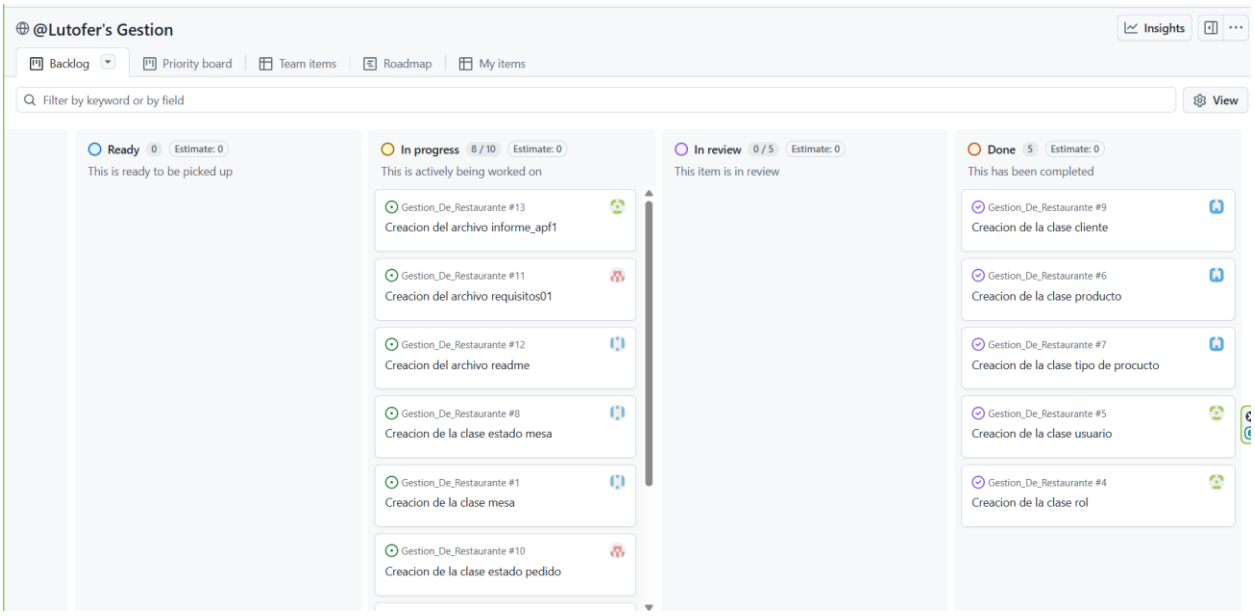


Figura 24. Tareas completadas en el tablero Kanban

(Fuente: Elaboración propia)

9.2.3. María Collana – Desarrollo de módulos Mesa y Estado de Mesa

Ramas utilizadas: feature/creacion-mesa-crud, feature/creacion-estado-mesa

Descripción proceso:

Paso 1: Creación de la rama feature

Se crea la rama correspondiente para el desarrollo del módulo mesa a partir de la rama develop.

Comando utilizado:

git checkout -b feature/creacion-mesa-crud

```
Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (develop)
$ git checkout -b feature/creacion-mesa-crud
Switched to a new branch 'feature/creacion-mesa-crud'

Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-mesa-crud)
$
```

Figura 25. Creación de la rama feature/creacion-mesa-crud

(Fuente: Elaboración propia)

Paso 2: Desarrollo de las funcionalidades

Se pasa a implementar las capas de Entity, Repository, Service y Controller de la clase Mesa, siguiendo con el modelo del proyecto MVC.

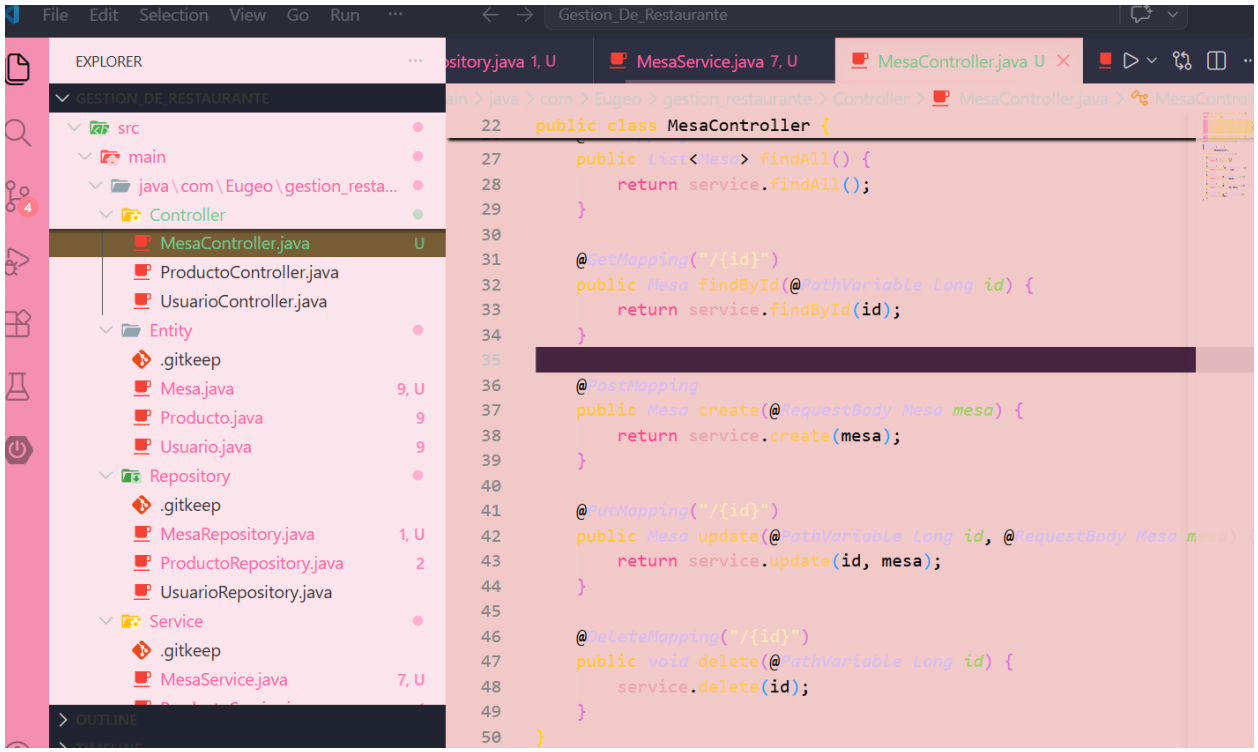


Figura 26. Implementación de los módulos de Mesa

(Fuente: Elaboración propia)

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

git add .

git commit -m “feat: Implementa CRUD de Mesa (entity, repository, service, controller) (Closes #1)”

git push -u origin feature/creacion-mesa-crud

```
Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-mesa-crud)
$ git commit -m "feat: Implementa CRUD de Mesa (entity, repository, services, co
ntroller) (Closes #1)"
[feature/creacion-mesa-crud f889a7f] feat: Implementa CRUD de Mesa (entity, repo
sitory, services, controller) (Closes #1)
 4 files changed, 174 insertions(+)
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Controller/MesaC
ontroller.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Entity/Mesa.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Repository/MesaR
epository.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Service/MesaServ
ice.java

Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-mesa-crud)
$ git push -u origin feature/creacion-mesa-crud
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 4 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (16/16), 2.92 KiB | 426.00 KiB/s, done.
Total 16 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/creacion-mesa-crud' on GitHub by visi
ting:
remote:      https://github.com/Lutofer/Gestion_De_Restaurante/pull/new/feature/
creacion-mesa-crud
remote:
To https://github.com/Lutofer/Gestion_De_Restaurante.git
 * [new branch]      feature/creacion-mesa-crud -> feature/creacion-mesa-crud
branch 'feature/creacion-mesa-crud' set up to track 'origin/feature/creacion-mes
a-crud'.
```

Figura 27. Registro de cambios en el repositorio

(Fuente: Elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

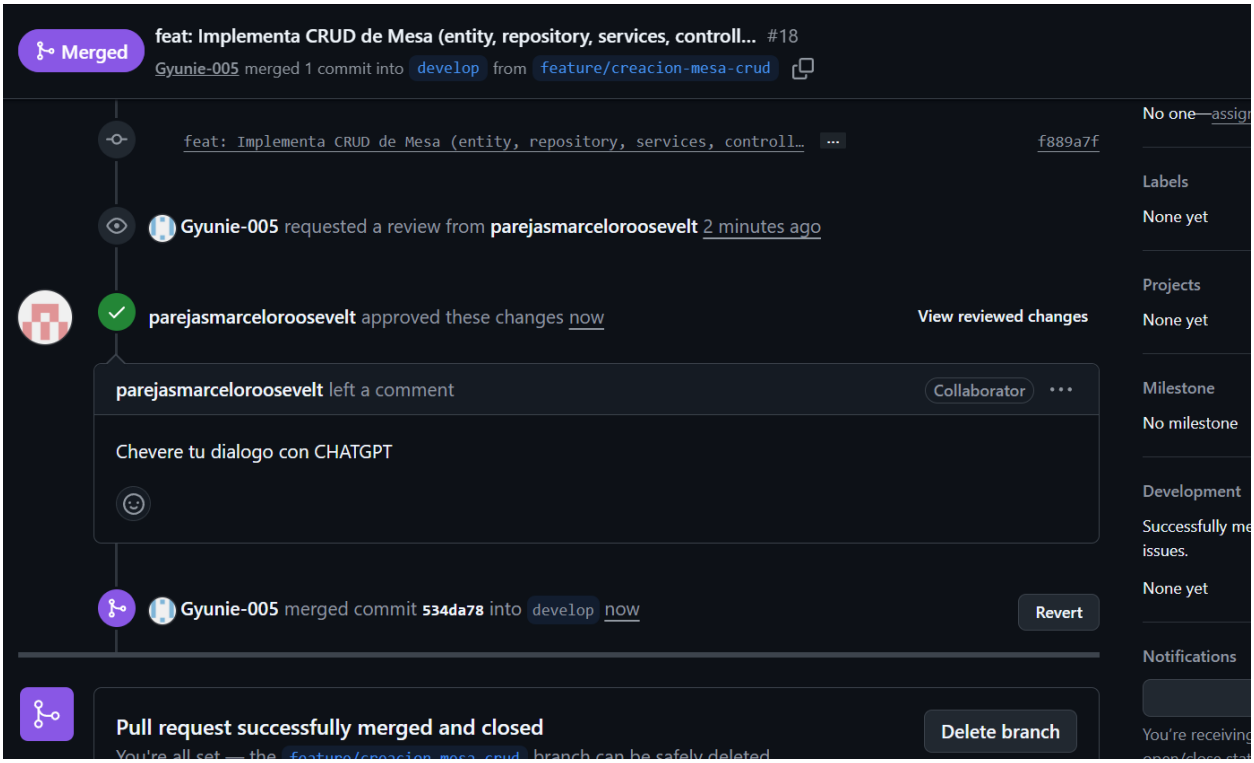


Figura 28. Merge de la rama feature hacia develop

(Fuente: Elaboración propia)

De igual manera se realiza con cada rama que se le asignó a Mafer de esta manera completa sus 2 Issues asignados en el tablero Kanban

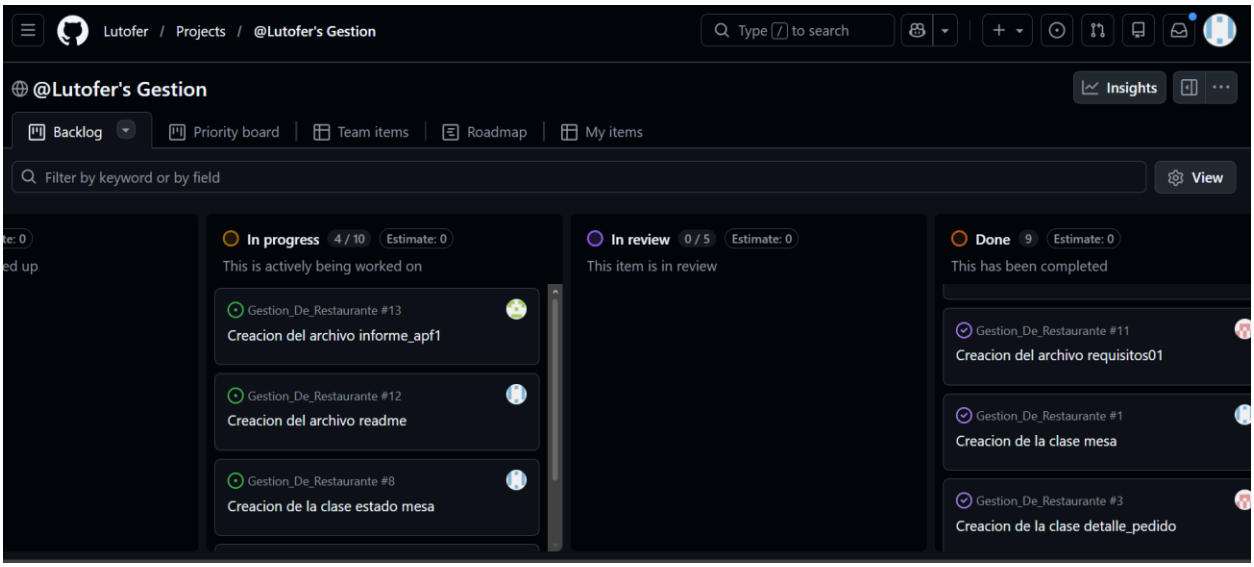


Figura 29. Tareas completadas en el tablero Kanban

(Fuente: Elaboración propia)

9.3. Ejecución de tareas asignadas subiendo respectivos documentación

9.3.1. Roosevelt Parejas – Subir requisitos_01

Ramas utilizadas: feature/creacion-requistos_01

Descripción del proceso:

Paso 1: Creación de la rama feature

Se crea la rama correspondiente para el desarrollo del módulo producto a partir de la rama main.

Comando utilizado:

git checkout -b feature/creacion-requisitos_01

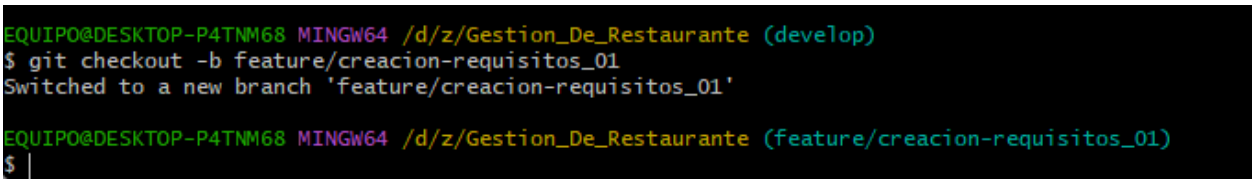


Figura 30. Creación de la rama feature/creacion-requisitos_01

(Fuente: Elaboración propia)

Paso 2: Se sube el informe al github

Se ha realizado la subida de los requisitos 01 donde contiene los datos que usaremos más adelante.

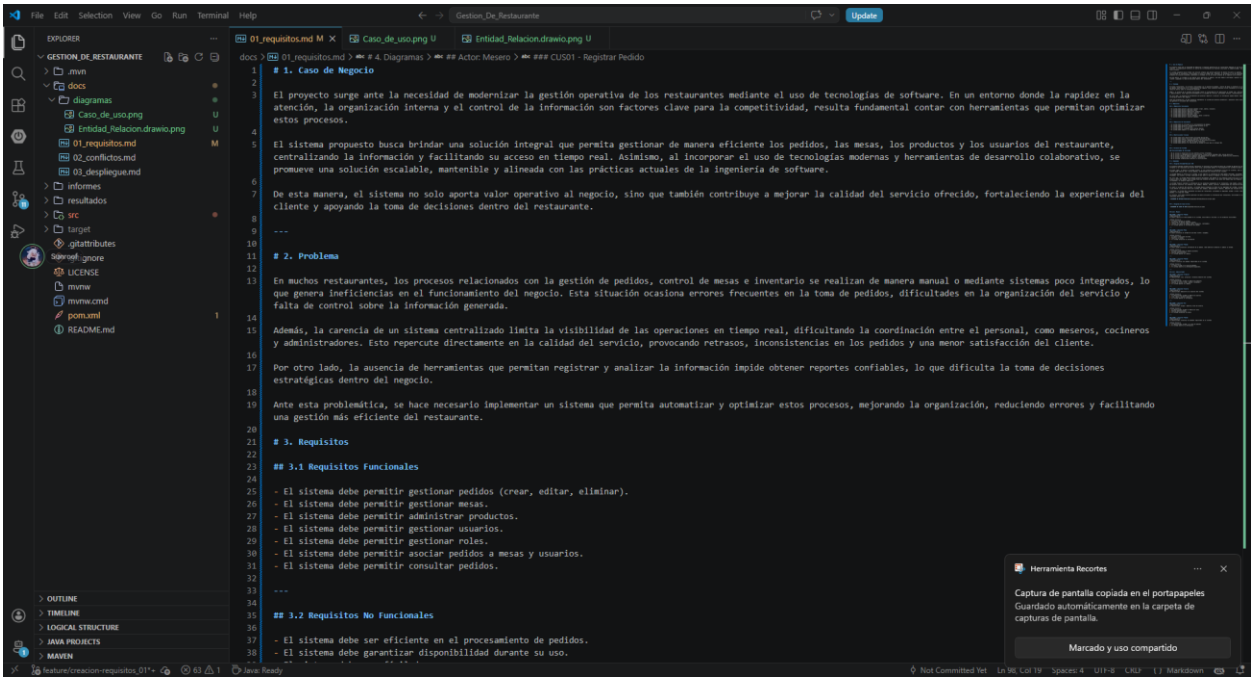


Figura 31. Redacción del informe y la creación de la carpeta Diagramas con sus respectivos elementos.

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

git add .

git commit -m "feat: Creación del archivo 01_requisitos (Closes #13)"

git push -u origin feature/creacion-requisitos_01

```
EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (develop)
$ git checkout -b feature/creacion-requisitos_01
Switched to a new branch 'feature/creacion-requisitos_01'

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-requisitos_01)
$ git add .

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-requisitos_01)
$ git commit -m "feat: Creación del archivo requisitos_01 (Closes #11)"
[feature/creacion-requisitos_01 256cc26] feat: Creación del archivo requisitos_01 (Closes #11)
 11 files changed, 567 insertions(+)
 create mode 100644 docs/diagramas/Caso_de_uso.png
 create mode 100644 docs/diagramas/Entidad_Relacion.drawio.png
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Controller/MesaController.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Controller/PedidoController.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Entity/Mesa.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Entity/Pedido.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Repository/MesaRepository.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Repository/PedidoRepository.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Service/MesaService.java
 create mode 100644 src/main/java/com/Eugeo/gestion_restaurante/Service/PedidoService.java

EQUIPO@DESKTOP-P4TNM68 MINGW64 /d/z/Gestion_De_Restaurante (feature/creacion-requisitos_01)
$ git puch -u origin feature/creacion-requisitos_01
git: 'puch' is not a git command. See 'git --help'.

remote:t similar command is
To https://github.com/Lutofer/Gestion_De_Restaurante.git
 * [new branch]      feature/creacion-requisitos_01 -> feature/creacion-requisitos_01
branch 'feature/creacion-requisitos_01' set up to track 'origin/feature/creacion-requisitos_01'.
$ git push -u origin feature/creacion-requisitos_01
Enumerating objects: 48, done.
Counting objects: 100% (48/48), done.
Delta compression using up to 16 threads
Compressing objects: 100% (29/29), done.
Writing objects: 100% (35/35), 127.74 KiB | 14.19 MiB/s, done.
Total 35 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 2 local objects.
remote:
remote: Create a pull request for 'feature/creacion-requisitos_01' on GitHub by visiting:
remote:      https://github.com/Lutofer/Gestion_De_Restaurante/pull/new/feature/creacion-requisitos_01
```

Figura 32. Se agrega los comandos al git bash

(Fuente: Elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

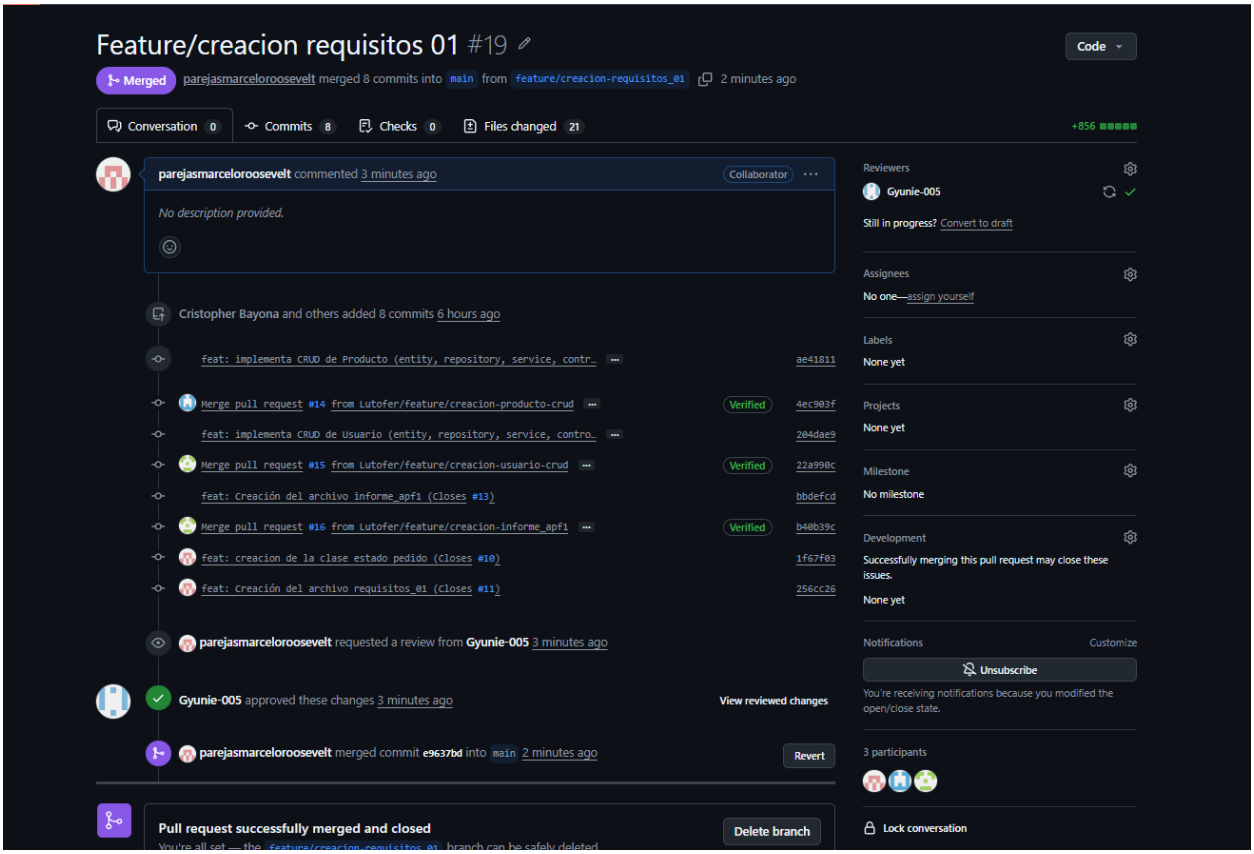


Figura 33. Merge de la rama feature hacia develop

(Fuente: Elaboración propia)

9.3.2. Loana Muñoz – Subir informe_apf1

Ramas utilizadas: feature/creacion-informe_apf1

Descripción del proceso:

Paso 1: Creación de la rama feature

Se crea la rama correspondiente para el desarrollo del módulo producto a partir de la rama main.

Comando utilizado:

git checkout -b feature/creacion-informe_apf1

```
ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (main)
$ git checkout -b feature/creacion-informe_apf1
Switched to a new branch 'feature/creacion-informe_apf1'

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-informe_apf1)
$
```

Figura 34. Creación de la rama feature/creacion-informe_apf1

(Fuente: Elaboración propia)

Paso 2: Se sube el informe al github

Se ha realizado la subida del informe apf1 donde contiene los datos que usaremos mas adelante

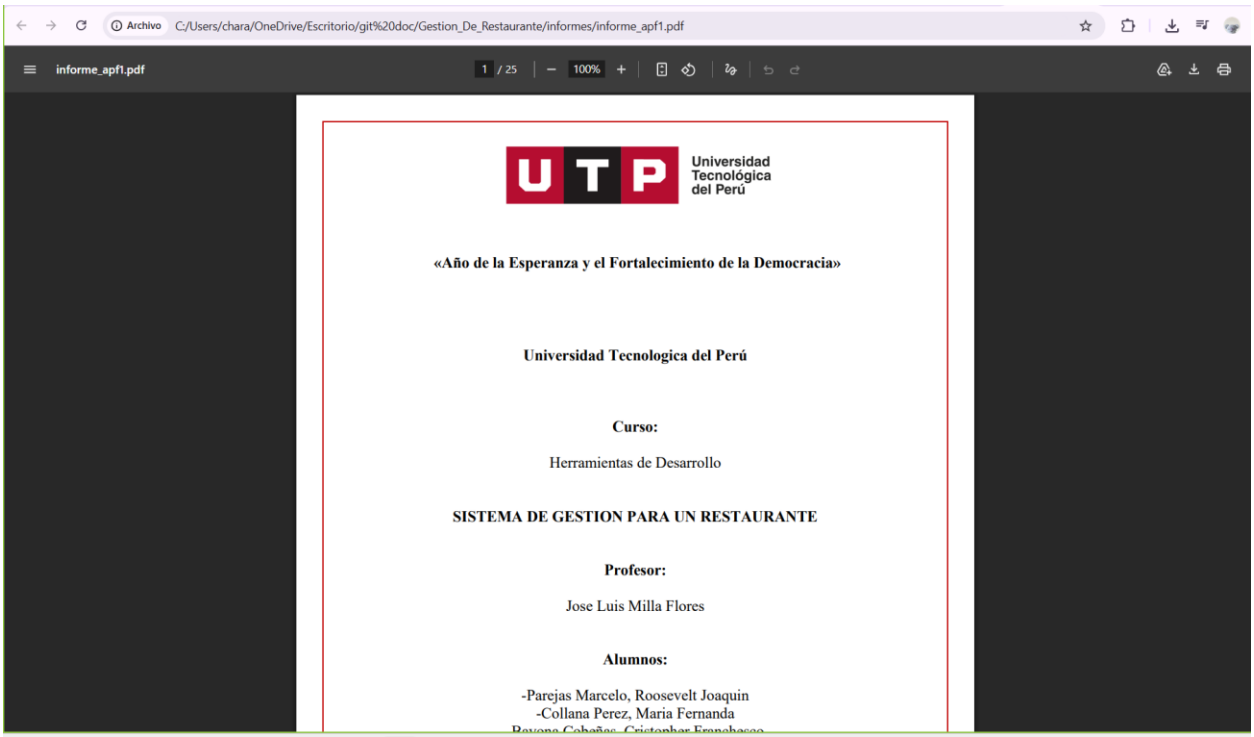


Figura 35. Agregar el informe_apf1 a la carpeta

(Fuente: Elaboración propia)

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

```
git add .
git commit -m "feat: Creación del archivo informe_apf1 (Closes #13)"
git push -u origin feature/creacion-informe_apf1
```

```
ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-informe_apf1)
$ git add .

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-informe_apf1)
$ git commit -m "feat: Creación del archivo informe_apf1 (Closes #13)"
[feature/creacion-informe_apf1 bbdefcd] feat: Creación del archivo informe_apf1
(Closes #13)
 1 file changed, 0 insertions(+), 0 deletions(-)

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-informe_apf1)
$ git push -u origin feature/creacion-informe_apf1
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 394.54 KiB | 20.76 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/creacion-informe_apf1' on GitHub by v
isiting:
remote:      https://github.com/Lutofer/Gestion_De_Restaurante/pull/new/feature/
creacion-informe_apf1
remote:
To https://github.com/Lutofer/Gestion_De_Restaurante.git
 * [new branch]      feature/creacion-informe_apf1 -> feature/creacion-informe_a
pf1
branch 'feature/creacion-informe_apf1' set up to track 'origin/feature/creacion-
informe_apf1'.

ASUS@DESKTOP-CFM5DOD MINGW64 ~/OneDrive/Escritorio/git doc/Gestion_De_Restaurant
e (feature/creacion-informe_apf1)
$
```

Figura 36. Se agrega los comandos al git bash

(Fuente: Elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

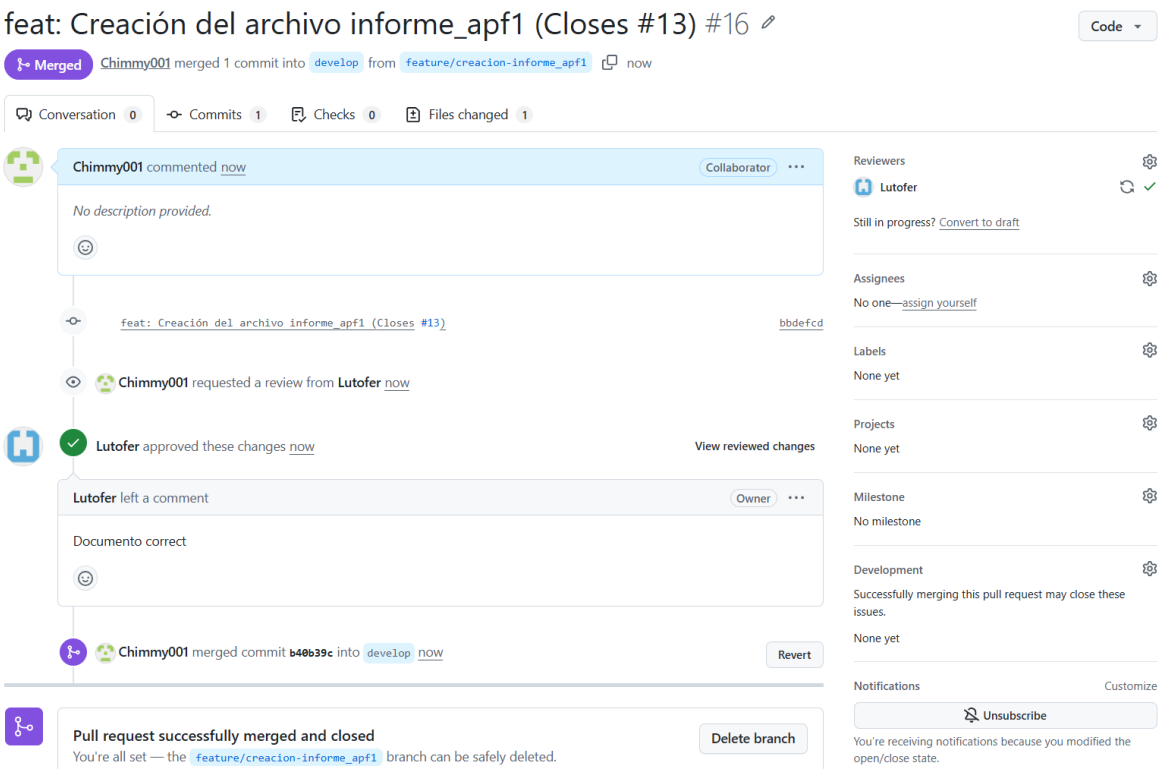


Figura 37. Merge de la rama feature hacia develop

(Fuente: Elaboración propia)

9.3.3. María Collana – Subir readme

Ramas utilizadas: feature/creacion-readme

Descripción del proceso:

Paso 1: Creación de la rama feature

Se crea la rama correspondiente para el desarrollo del módulo producto a partir de la rama main.

Comando utilizado:

git checkout -b feature/creacion-readme

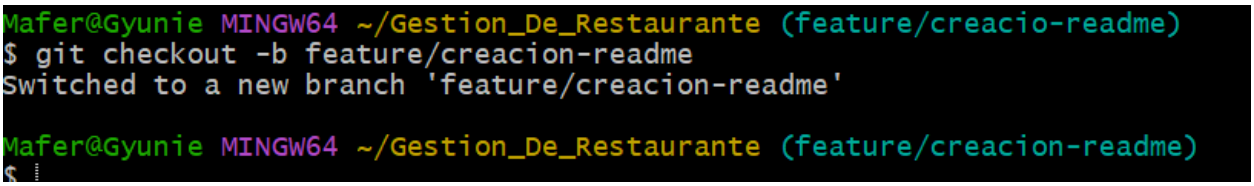


Figura 38. Creación de la rama feature/creacion-readme

(Fuente: Elaboración propia)

Paso 2: Se sube el informe al github

Se ha realizado la subida de README donde contiene los datos que usaremos más adelante.

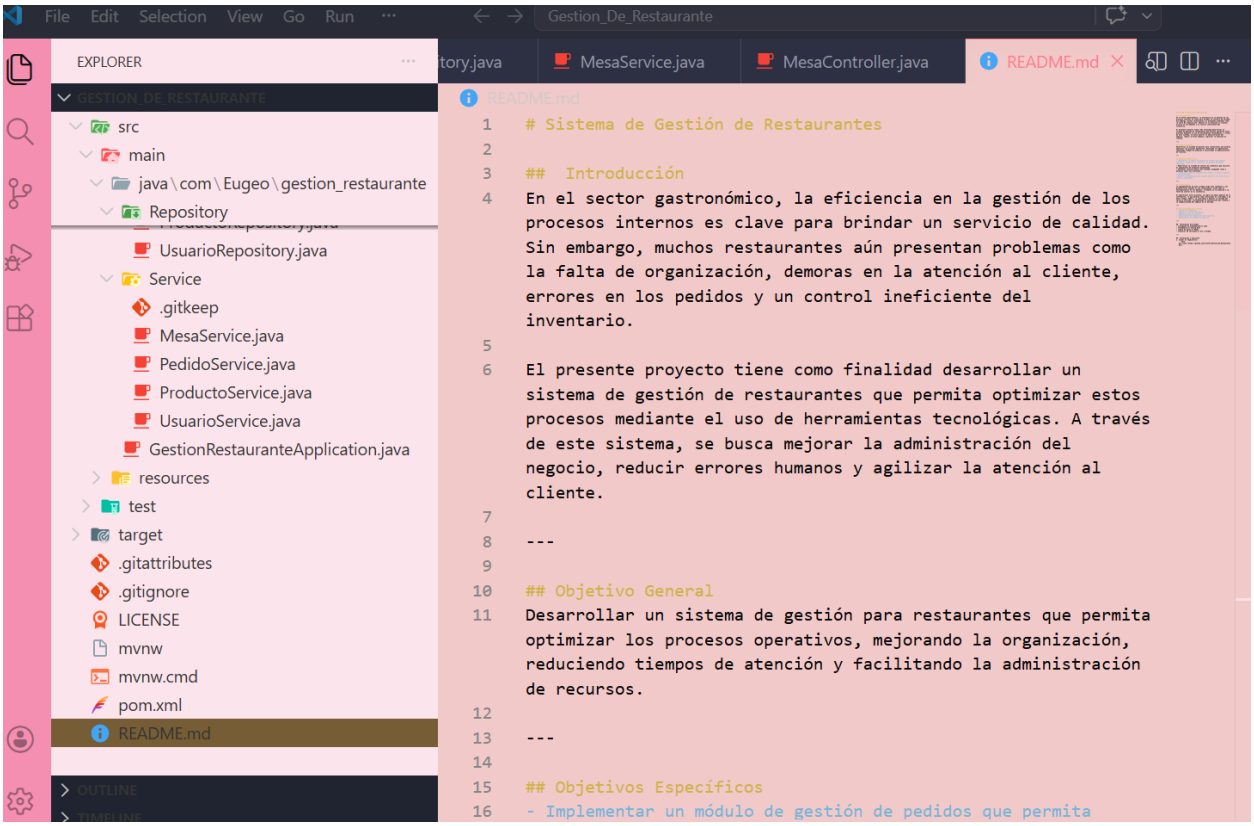


Figura 39. Redacción del README

(Fuente: Elaboración propia)

Paso 3: Registro de cambios

Se agregan los archivos modificados al área de preparación y se realiza el commit correspondiente.

Comandos utilizados:

```
git add .

git commit -m "feat: Creación del archivo README (Closes #12)"

git push -u origin feature/creacion-readme
```

```
Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-readme)
$ git add .

Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-readme)
$ git add .

Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-readme)
$ git commit -m "feat: Creación de archivo README (Closes #12)"
[feature/creacion-readme b5ed479] feat: Creación de archivo README (Closes #12)
 1 file changed, 50 insertions(+)

Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-readme)
$ git push -u origin feature/creacion-readme
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.38 KiB | 704.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/creacion-readme' on GitHub by visitin
g:
remote:      https://github.com/Lutofer/Gestion_De_Restaurante/pull/new/feature/
creacion-readme
remote:
To https://github.com/Lutofer/Gestion_De_Restaurante.git
 * [new branch]      feature/creacion-readme -> feature/creacion-readme
branch 'feature/creacion-readme' set up to track 'origin/feature/creacion-readme
'.

Mafer@Gyunie MINGW64 ~/Gestion_De_Restaurante (feature/creacion-readme)
```

Figura 40. Se agregan los comando al git bash

(Fuente: Elaboración propia)

Paso 4: Integración a la rama develop

Luego de la aprobación correspondiente en el Pull Request de parte de la persona a la cual se le asignó de modo de Reviewer, se realiza el merge de la rama feature hacia develop.

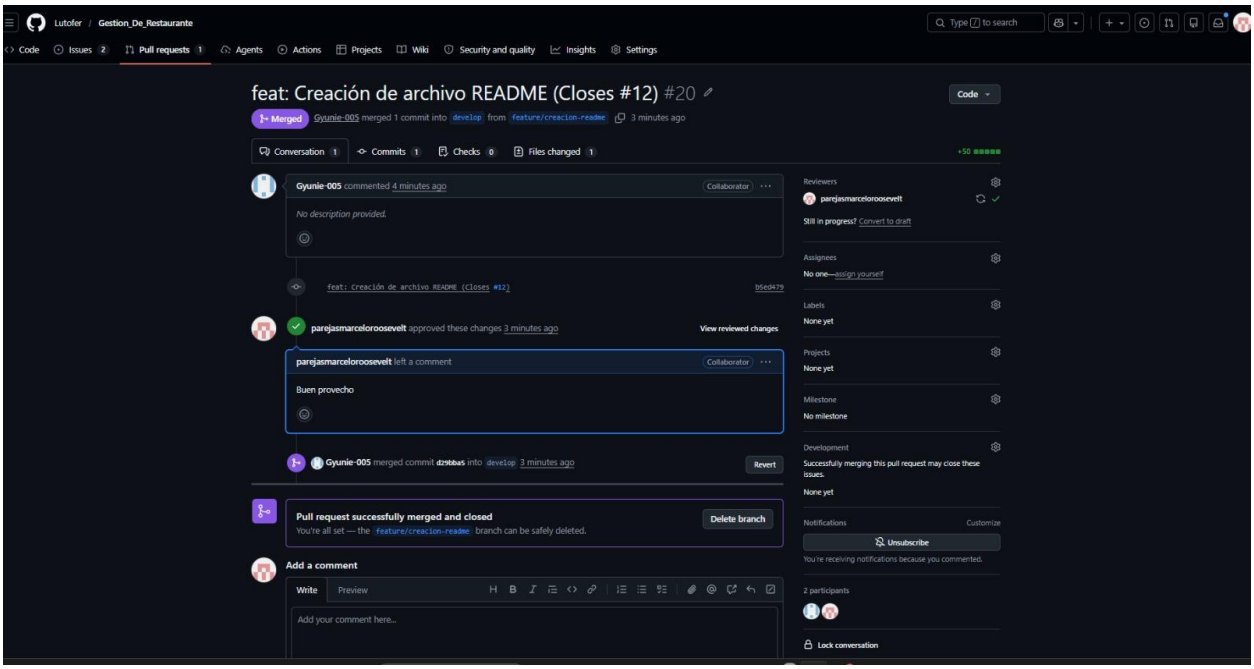


Figura 41. Merge de la rama feature hacia develop

(Fuente: Elaboración propia)

10. Bibliografias:

Cruz Ulloa, M. (2024). Gestión de inventarios y su impacto en la rentabilidad de restaurantes. Revista de Investigación en Ciencias Sociales. Recuperado de: https://revistas-facet-unc.edu.py/index.php/RICS/article/view/49

Restrepo, J., et al. (2020). Diseño de un sistema de gestión de inventario para un restaurante. ResearchGate. Recuperado de: https://www.researchgate.net/publication/382719545_Diseno_de_un_sistema_de_gestion_de_inventario_para_un_restaurante_manabita

Álvarez Soto, F., & Cisternas Aranda, P. (2020). Propuesta de mejora de procesos en restaurantes mediante sistemas tecnológicos. Universidad Técnica Federico Santa María. Recuperado de: https://repositorio.usm.cl/entities/tesis/e3a94266-0113-4287-acc9-79f4bca05270

Pinos Robles, L. (2021). Implementación de sistemas informáticos en la gestión de restaurantes. Pontificia Universidad Católica del Ecuador. Recuperado de: https://repositorio.puce.edu.ec/handle/123456789/27376

Oracle Hospitality. (s.f.). Restaurant technology solutions. Oracle. https://www.oracle.com/food-beverage/

Cárdenas, J., & colaboradores. (2023). Sistema de información para dinamizar la gestión comercial de un restaurante con metodología Scrum. Revista Científica INGENIAR. Recuperado de https://journalingeniar.org/index.php/ingeniar/article/view/167

Universidad Nacional de San Martín. (2023). Sistema web para la dinamización de la gestión de inventarios y estrategias de comercialización en supermercados peruanos. Revista Científica de Sistemas e Informática. Recuperado de

https://revistas.unsm.edu.pe/index.php/rcsi/article/view/673

Cornide Reyes, H. et al. (2025). Mejora de procesos de software en equipos Scrum: Una revisión de la literatura. Recuperado de https://www.scielo.cl/scielo.php?pid=S0718-33052025000100210&script=sci_arttext

Pardo-Calvache, C. et al. (2019). Scrum+: Un Scrum escalado para la gestión de proyectos ágiles. Recuperado de https://www.redalyc.org/journal/430/43062836010/html/

VersionOne / Journal of Systems and Software (2022). Por qué y cómo se está adaptando Scrum en la práctica: una revisión sistemática. Recuperado de https://www.sciencedirect.com/science/article/abs/pii/S0164121221002077